Namespace LlamaLogic.TuningArchive

Classes

GameInstallation
: Represents an installation of The Sims 4 from which a TuningArchiveFile can capture a snapshot (🔒)

GameInstallationPack
: Represents the base game or a pack in a GameInstallation (🔒)

GameInstallationPackage
: Represents a package file of a GameInstallationPack and the priority the game loads it with

Sims4StudioBatchImport
: Names files the way Sims 4 Studio's batch import reads them

TuningAnalysis
: Represents a tuning document understood through a set of tuning descriptions

TuningArchiveCaptureOptions
: Represents the choices a creator makes when capturing a snapshot from an installation of the game

TuningArchiveCaptureResult
: Represents the outcome of capturing or importing a snapshot into a TuningArchiveFile

TuningArchiveChange
: Represents how a resource differs, if at all, between two snapshots

TuningArchiveDescriptions
: Represents a set of tuning descriptions (TDESCs) imported into a TuningArchiveFile

TuningArchiveEntry
: Represents a version of a resource in a TuningArchiveFile

TuningArchiveExportOptions
: Represents the choices a creator makes when exporting a snapshot as files, the way Scumbumbo's XML Extractor extracts the game's tuning

TuningArchiveExportResult
: Represents the outcome of exporting a snapshot as files

TuningArchiveFile
: Represents an archive of The Sims 4's tuning across versions of the game, stored in a SQLite database file (🔒)

TuningArchiveHistoryItem

Represents a version of a resource and the snapshots that contain it

TuningArchiveImportOptions

Represents the choices a creator makes when importing a folder of tuning files extracted by another tool as a snapshot

TuningArchiveReference

Represents a reference from a tuning to another tuning or resource

TuningArchiveSnapshot

Represents the state of the game's tuning at a point in time, as captured in a TuningArchiveFile

TuningArchiveSnapshotPack

Represents a pack that was present when a TuningArchiveSnapshot was taken

TuningArchiveStringChange

Represents how a string differs, if at all, between two snapshots

TuningArchiveUninstalledTuning

Represents a tuning of a pack that was not installed when a TuningArchiveSnapshot was captured, known only from the pack's patches in the game's `Delta` folder

TuningCommenter

Adds to tuning the comments XML Extractor adds, as a snapshot knows them: the text of each string after its key, and the class and name of the tuning or animation state machine each reference refers to

TuningCoreLibraryDraft

Drafts a Lot 51 Core Library tuning injector snippet (for Core Library 1.43) that makes changes to tuning without overriding it

TuningDescriptionCache

Keeps the documents of a TuningDescriptionSet that analysis has used, so that analyzing many tunings does not read and parse the same descriptions repeatedly (🔒)

TuningDraftDiagnostic

Represents what a draft does, or cannot do, about a change to a tunable

TuningDraftResult

Represents a draft that makes changes to tuning without overriding it: a script or a snippet

TuningDraftTarget

Represents a tuning a creator has changed in a draft: the tuning as the archive has it and as the creator wants it

TuningFolderUpdate

Finds the overrides of game tuning in every package in a folder and its subfolders, works out what a game version means for each, checks the SimData of the tuning the packages add against the game version's tuning descriptions, and writes updated copies of the packages with the changes that need no judgment from the creator, with a report of everything

TuningFolderUpdateOptions

Represents which changes a folder update makes beyond the merges that need no judgment from the creator

TuningFolderUpdatePackage

Represents a package in a folder being updated

TuningFolderUpdateReportRow

Represents a row of a folder update's report: an override, the SimData of a tuning a package adds, or a package that couldn't be read

TuningFolderUpdateResult

Represents what writing a folder update did

TuningFolderUpdateWrite

Represents what writing a folder update did for one of its packages

TuningMerge

Merges a mod's changes to a tuning onto the game's new version of it: a three-way merge of the version the mod was made from (the base), the mod's version, and the game's new version

TuningMergeConflict

Represents a place where a merge could not combine the mod's change with the game's

TuningMergeResult

Represents the result of a merge of a mod's changes onto the game's new version of a tuning

TuningNode

Represents an element of a tuning document, as understood through its tuning description

TuningPackageAddedSimData

Represents the SimData of tuning a package adds (rather than overrides), checked against SimData generated from the tuning with the tuning descriptions of the game version the package is being updated for

TuningPackageAddedSimDataPlan

Represents what a set of fixes does to the SimData of tuning a package adds, and to the tuning

TuningPackageOverride
: Represents a mod's override of a game tuning, and what a game version means for it

TuningPackageUpdate
: Finds a mod package's overrides of game tuning, works out what a game version means for each, and writes an updated copy of the package

TuningPackageUpdateChange
: Represents what writing an updated package did to a resource

TuningPythonDraft
: Drafts a Python script mod that makes changes to tuning as the game loads it, without overriding any tuning

TuningSemanticChange
: Represents an element that differs between two versions of a tuning document

TuningSemanticDiff
: Compares two versions of a tuning document element by element, matching elements by name and aligning list items, rather than line by line

TuningSimDataDraftResult
: Represents the SimData an override of a tuning needs

TuningSimDataDrafter
: Generates the SimData that overrides of a snapshot's tuning need, trusting the result only when the generator reproduces the game's own SimData for the original tuning

TuningXml
: Provides the canonical text form in which a tuning archive stores tuning and SimData documents

TuningXmlInjectorDraft
: Drafts an XML Injector snippet (for Scumbumbo's XML Injector, version 4) that makes changes to tuning without overriding it

Structs

TuningArchiveProgress
: Represents the progress of a long-running TuningArchiveFile operation

Enums

TuningArchiveChangeKind
: Specifies how a resource differs, if at all, between two snapshots

TuningArchiveExportAttributeOrder
: Specifies the order in which ExportAsync(TuningArchiveSnapshot, string, TuningArchiveExportOptions?, IProgress<TuningArchiveProgress>?, CancellationToken) writes the `n` and `t` attributes of an element that has both

TuningArchiveExportExistingFiles
: Specifies what ExportAsync(TuningArchiveSnapshot, string, TuningArchiveExportOptions?, IProgress<TuningArchiveProgress>?, CancellationToken) does when a file it would write already exists

TuningArchiveExportNaming
: Specifies how ExportAsync(TuningArchiveSnapshot, string, TuningArchiveExportOptions?, IProgress<TuningArchiveProgress>?, CancellationToken) names the files it writes, following the conventions of Scumbumbo's XML Extractor

TuningArchiveResourceKind
: Specifies the kind of a resource in a TuningArchiveFile

TuningArchiveSnapshotSource
: Specifies where the contents of a TuningArchiveSnapshot came from

TuningDraftSeverity
: Specifies what a draft can do about a change

TuningMergeChoice
: Specifies which side of a merge conflict a merge takes

TuningMergeConflictKind
: Specifies why a merge could not combine the mod's change with the game's

TuningPackageAddedSimDataFixes
: Specifies which fixes writing an updated package makes to the SimData of tuning it adds, and to that tuning

TuningPackageAddedSimDataProblems
: Specifies what is wrong with the SimData of tuning a package adds, by the tuning descriptions of the game version it is being updated for

TuningPackageOverrideOutcome
: Specifies what a game version means for a mod's override of a tuning

TuningPackageSimDataChanges
: Specifies which changes writing an updated package makes to its SimData

TuningPackageSimDataState

Specifies whether a mod's SimData for an override is what the override's tuning needs in the game's new version

TuningPackageUpdateChangeKind

Specifies what writing an updated package did to a resource

TuningSemanticChangeKind

Specifies how an element differs between two versions of a tuning document

TuningSimDataDraftKind

Specifies what an override of a tuning needs from its SimData

TuningValueKind

Specifies what the value of an element of a tuning document means, according to its tuning description

Class GameInstallation

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents an installation of The Sims 4 from which a TuningArchiveFile can capture a snapshot (🔒)

```
public sealed class GameInstallation
```

Inheritance

object ← GameInstallation

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

ContentRoot

Gets the folder containing the installation's `Data` folder

```
public string ContentRoot { get; }
```

Property Value

string

GameVersion

Gets the version of the game, as the installation's configuration states it, if it could be found

```
public string? GameVersion { get; }
```

Property Value

string

Locales

Gets the locales for which the base game has string tables (such as `ENG_US`)

```
public IReadOnlyCollection<string> Locales { get; }
```

Property Value

IReadOnlyCollection <string>

Packs

Gets the base game (always first) and the packs that are installed

```
public IReadOnlyList<GameInstallationPack> Packs { get; }
```

Property Value

IReadOnlyList <GameInstallationPack>

Path

Gets the path with which the installation was opened

```
public string Path { get; }
```

Property Value

string

UninstalledPacks

Gets the packs that are not installed but have patches in the installation's `Delta` folder, which the game does not load

```
public IReadOnlyList<GameInstallationPack> UninstalledPacks { get; }
```

Property Value

IReadOnlyList<GameInstallationPack>

Methods

FindInstallations()

Finds the installations of The Sims 4 in the places the game's installers usually put it and, on macOS, wherever Spotlight finds its app bundle

```
public static IReadOnlyList<string> FindInstallations()
```

Returns

IReadOnlyList<string>

Open(string)

Opens the installation of The Sims 4 at the specified `path`, which may be the installation folder, the app bundle on macOS, or the folder containing it

```
public static GameInstallation Open(string path)
```

Parameters

`path` string

Returns

GameInstallation

Remarks

On macOS, packs are read from the `The Sims 4 Packs` folder beside the app bundle, where the game installs them

Exceptions

DirectoryNotFoundException

> No installation of The Sims 4 was found at `path`

Class GameInstallationPack

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents the base game or a pack in a GameInstallation (🔒)

```
public sealed class GameInstallationPack
```

Inheritance

object ← GameInstallationPack

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object)

Properties

ClientPackages

Gets the packages of the pack containing client resources other than string tables (such as images, sounds and videos), in descending order of priority

```
public IReadOnlyList<GameInstallationPackage> ClientPackages { get; }
```

Property Value

IReadOnlyList<GameInstallationPackage>

Code

Gets the code of the pack (`BG` for the base game, otherwise the name of the pack's folder, such as `EP01`)

```
public string Code { get; }
```

Property Value

string

Locales

Gets the locales for which the pack has string tables (such as `ENG_US`)

```
public IReadOnlyCollection<string> Locales { get; }
```

Property Value

IReadOnlyCollection <string>

SimulationPackages

Gets the packages of the pack containing simulation resources (combined tuning and SimData), in descending order of priority

```
public IReadOnlyList<GameInstallationPackage> SimulationPackages { get; }
```

Property Value

IReadOnlyList <GameInstallationPackage>

Version

Gets the version of the pack, as its version file states it, if it has one

```
public string? Version { get; }
```

Property Value

string

Methods

GetStringTablePackages(string)

Gets the packages of the pack containing string tables for the specified `locale`, in descending order of priority

```
public IReadOnlyList<GameInstallationPackage> GetStringTablePackages(string locale)
```

Parameters

`locale` string

Returns

IReadOnlyList<GameInstallationPackage>

ToString()

Returns a string that represents the current object.

```
public override string ToString()
```

Returns

string

A string that represents the current object.

Class GameInstallationPackage

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a package file of a GameInstallationPack and the priority the game loads it with

```
public sealed record GameInstallationPackage : IEquatable<GameInstallationPackage>
```

Inheritance

object ← GameInstallationPackage

Implements

IEquatable<GameInstallationPackage>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

GameInstallationPackage(string, int, bool)

Represents a package file of a GameInstallationPack and the priority the game loads it with

```
public GameInstallationPackage(string Path, int Priority, bool IsPatch = false)
```

Parameters

`Path` string

The path of the package file

`Priority` int

The priority the game's resource configuration assigns the package; when packages contain the same resource, the one with the greater priority prevails

`IsPatch` bool

Whether the package is in the pack's folder under `Delta`, which patches the files installed with the pack

Properties

IsPatch

Whether the package is in the pack's folder under `Delta`, which patches the files installed with the pack

```
public bool IsPatch { get; init; }
```

Property Value

bool

Path

The path of the package file

```
public string Path { get; init; }
```

Property Value

string

Priority

The priority the game's resource configuration assigns the package; when packages contain the same resource, the one with the greater priority prevails

```
public int Priority { get; init; }
```

Property Value

int

Class Sims4StudioBatchImport

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Names files the way Sims 4 Studio's batch import reads them

```
public static class Sims4StudioBatchImport
```

Inheritance

object ← Sims4StudioBatchImport

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Batch import reads a resource's key from the start of its file name, up to the first period: the type, group and instance in hexadecimal, separated by exclamation points. Sims 4 Studio's batch export follows the key with the resource's name and type, for people, as in `6017E896!00000000!0000000000012345.buff_Example.BuffTuning.xml`, and so do these names. Characters Windows doesn't allow in file names become underscores, as they do in Sims 4 Studio's export.

Methods

GetFileName(ResourceKey, string?)

Gets the name of a file of XML for the resource with the specified `key` and `name`

```
public static string GetFileName(ResourceKey key, string? name)
```

Parameters

`key` ResourceKey

`name` string

Returns

string

GetFileName(string)

Gets the name of a file of the specified tuning document `text`, whose key comes from its root element: the type from its `i` attribute, the instance from its `s` attribute, and the group 0

```
public static string GetFileName(string text)
```

Parameters

`text` string

Returns

string

Exceptions

ArgumentException

The root element does not have both attributes, or its instance is not a number

Class TuningAnalysis

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a tuning document understood through a set of tuning descriptions

```
public sealed class TuningAnalysis
```

Inheritance

object ← TuningAnalysis

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Instance

Gets the instance description of the document, if it is instance tuning and its class was found

```
public TuningDescriptionInstance? Instance { get; }
```

Property Value

TuningDescriptionInstance

InstanceMembers

Gets the tunables the document's class has, including those it inherits, in the order the game collects them, if it is instance tuning and its class was found; otherwise, an empty list

```
public IReadOnlyList<TunableDescription> InstanceMembers { get; }
```

Property Value

IReadOnlyList<TunableDescription>

Module

Gets the module description of the document, if it is module tuning and its module was found

```
public TuningDescriptionModule? Module { get; }
```

Property Value

TuningDescriptionModule

Nodes

Gets the elements of the document, in document order

```
public IReadOnlyList<TuningNode> Nodes { get; }
```

Property Value

IReadOnlyList<TuningNode>

PythonModule

Gets the Python module the document's class is in (its `m` attribute), or the module of module tuning

```
public string? PythonModule { get; }
```

Property Value

string

References

Gets the references in the document

```
public IEnumerable<TuningNode> References { get; }
```

Property Value

IEnumerable <TuningNode>

UnrecognizedPaths

Gets the elements whose names the tuning descriptions do not recognize, by path (a large number suggests the descriptions are for another version of the game)

```
public IReadOnlyList<string> UnrecognizedPaths { get; }
```

Property Value

IReadOnlyList <string>

Methods

Analyze(string, TuningDescriptionCache)

Analyzes the specified tuning document `text` (canonical text, see TuningXml) with the tuning descriptions of the specified `cache`, which keeps the descriptions it reads for later analyses

```
public static TuningAnalysis Analyze(string text, TuningDescriptionCache cache)
```

Parameters

`text` string

`cache` TuningDescriptionCache

Returns

TuningAnalysis

Analyze(string, TuningDescriptionSet)

Analyzes the specified tuning document `text` (canonical text, see TuningXml) with the specified `descriptions`

```
public static TuningAnalysis Analyze(string text, TuningDescriptionSet descriptions)
```

Parameters

`text` string

`descriptions` TuningDescriptionSet

Returns

TuningAnalysis

GetReferenceResourceType(string)

Gets the resource type a tuning description's reference type name (such as `buff` or `caspart`) refers to

```
public static ResourceType GetReferenceResourceType(string typeName)
```

Parameters

`typeName` string

Returns

ResourceType

IsTuningReferenceType(string)

Gets whether a tuning description's reference type name refers to tuning (rather than another kind of resource)

```
public static bool IsTuningReferenceType(string typeName)
```

Parameters

`typeName` string

Returns

bool

Class TuningArchiveCaptureOptions

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents the choices a creator makes when capturing a snapshot from an installation of the game

```
public sealed class TuningArchiveCaptureOptions
```

Inheritance

object ← TuningArchiveCaptureOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

GameVersion

Gets/sets the game version to record, overriding the version the installation states (required when the installation's version cannot be found)

```
public string? GameVersion { get; set; }
```

Property Value

string

IncludeAnimationStateMachines

Gets/sets whether to capture animation state machines (the default is true)

```
public bool IncludeAnimationStateMachines { get; set; }
```

Property Value

bool

IncludeImages

Gets/sets whether to capture the images to which the captured tuning and SimData refer (the default is true)

```
public bool IncludeImages { get; set; }
```

Property Value

bool

IncludeSimData

Gets/sets whether to capture SimData resources (the default is true)

```
public bool IncludeSimData { get; set; }
```

Property Value

bool

IncludeSounds

Gets/sets whether to capture the sounds to which the captured tuning and SimData refer, with the audio they use (the default is false, because the audio is large and does not compress)

```
public bool IncludeSounds { get; set; }
```

Property Value

bool

IncludeVideos

Gets/sets whether to capture the videos to which the captured tuning and SimData refer (the default is false, because they are large and compress little)

```
public bool IncludeVideos { get; set; }
```

Property Value

bool

Label

Gets/sets a label for the snapshot

```
public string? Label { get; set; }
```

Property Value

string

Locale

Gets/sets the locale of the strings to capture, or null to capture no strings (the default is `ENG_US`)

```
public string? Locale { get; set; }
```

Property Value

string

Notes

Gets/sets notes to record with the snapshot

```
public string? Notes { get; set; }
```

Property Value

string

Packs

Gets/sets the codes of the packs to capture, or null to capture all installed packs (the default)

```
public IReadOnlyCollection<string>? Packs { get; set; }
```

Property Value

IReadOnlyCollection <string>

Class TuningArchiveCaptureResult

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents the outcome of capturing or importing a snapshot into a TuningArchiveFile

```
public sealed class TuningArchiveCaptureResult
```

Inheritance

object ← TuningArchiveCaptureResult

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AnimationStateMachines

Gets how many animation state machines the snapshot contains

```
public int AnimationStateMachines { get; }
```

Property Value

int

Duration

Gets how long the operation took

```
public TimeSpan Duration { get; }
```

Property Value

TimeSpan

Images

Gets how many images the snapshot contains

```
public int Images { get; }
```

Property Value

int

NewContent

Gets how many documents the archive did not already contain

```
public int NewContent { get; }
```

Property Value

int

NewContentBytes

Gets how many bytes the archive's documents grew by, compressed

```
public long NewContentBytes { get; }
```

Property Value

long

Problems

Gets descriptions of content that could not be captured, and of anything else the creator should know about the snapshot

```
public IReadOnlyList<string> Problems { get; }
```

Property Value

IReadOnlyList<string>

SimData

Gets how many SimData resources the snapshot contains

```
public int SimData { get; }
```

Property Value

int

Snapshot

Gets the snapshot that was created

```
public TuningArchiveSnapshot Snapshot { get; }
```

Property Value

TuningArchiveSnapshot

Sounds

Gets how many sounds the snapshot contains (audio configurations and the audio they use)

```
public int Sounds { get; }
```

Property Value

int

Strings

Gets how many strings the snapshot contains

```
public int Strings { get; }
```

Property Value

int

Tunings

Gets how many tunings the snapshot contains

```
public int Tunings { get; }
```

Property Value

int

Videos

Gets how many videos the snapshot contains

```
public int Videos { get; }
```

Property Value

int

Class TuningArchiveChange

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents how a resource differs, if at all, between two snapshots

```
public sealed record TuningArchiveChange : IEquatable<TuningArchiveChange>
```

Inheritance

object ← TuningArchiveChange

Implements

IEquatable<TuningArchiveChange>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningArchiveChange(TuningArchiveChangeKind, TuningArchiveEntry?, TuningArchiveEntry?)

Represents how a resource differs, if at all, between two snapshots

```
public TuningArchiveChange(TuningArchiveChangeKind Kind, TuningArchiveEntry? Old,
TuningArchiveEntry? New)
```

Parameters

`Kind` TuningArchiveChangeKind

How the resource differs

`Old` TuningArchiveEntry

The version of the resource in the older snapshot, if it was there

`New` TuningArchiveEntry

The version of the resource in the newer snapshot, if it is there

Properties

Entry

Gets the newer version of the resource if there is one, otherwise the older one

```
public TuningArchiveEntry Entry { get; }
```

Property Value

TuningArchiveEntry

Kind

How the resource differs

```
public TuningArchiveChangeKind Kind { get; init; }
```

Property Value

TuningArchiveChangeKind

New

The version of the resource in the newer snapshot, if it is there

```
public TuningArchiveEntry? New { get; init; }
```

Property Value

TuningArchiveEntry

Old

The version of the resource in the older snapshot, if it was there

```
public TuningArchiveEntry? Old { get; init; }
```

Property Value

TuningArchiveEntry

Enum TuningArchiveChangeKind

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies how a resource differs, if at all, between two snapshots

```
public enum TuningArchiveChangeKind
```

Fields

`Added = 0`

The resource is only in the newer snapshot

`Modified = 2`

The resource's content differs between the snapshots

`Moved = 3`

The resource's content is the same, but it moved to another pack or was renamed

`Removed = 1`

The resource is only in the older snapshot

`Unchanged = 4`

The resource is the same in both snapshots

Class TuningArchiveDescriptions

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a set of tuning descriptions (TDESCs) imported into a TuningArchiveFile

```
public sealed record TuningArchiveDescriptions : IEquatable<TuningArchiveDescriptions>
```

Inheritance

object ← TuningArchiveDescriptions

Implements

IEquatable<TuningArchiveDescriptions>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningArchiveDescriptions(long, string, string?, DateTimeOffset)

Represents a set of tuning descriptions (TDESCs) imported into a TuningArchiveFile

```
public TuningArchiveDescriptions(long Id, string GameVersion, string? FileName,
DateTimeOffset ImportedAt)
```

Parameters

`Id` long

The identifier of the set within its archive

`GameVersion` string

The version of the game the set describes

`FileName` string

The name of the file the set was imported from

`ImportedAt` DateTimeOffset

When the set was imported

Properties

FileName

The name of the file the set was imported from

```
public string? FileName { get; init; }
```

Property Value

string

GameVersion

The version of the game the set describes

```
public string GameVersion { get; init; }
```

Property Value

string

Id

The identifier of the set within its archive

```
public long Id { get; init; }
```

Property Value

long

ImportedAt

When the set was imported

```
public DateTimeOffset ImportedAt { get; init; }
```

Property Value

DateTimeOffset

Class TuningArchiveEntry

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a version of a resource in a TuningArchiveFile

```
public sealed class TuningArchiveEntry
```

Inheritance

object ← TuningArchiveEntry

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object)

Properties

Class

Gets the Python class of the tuning (its `c` attribute), if it is instance tuning

```
public string? Class { get; }
```

Property Value

string

Group

Gets the group of the resource

```
public uint Group { get; }
```

Property Value

uint

Instance

Gets the instance of the resource

```
public ulong Instance { get; }
```

Property Value

ulong

Key

Gets the resource key of the resource

```
public ResourceKey Key { get; }
```

Property Value

ResourceKey

Kind

Gets the kind of the resource

```
public TuningArchiveResourceKind Kind { get; }
```

Property Value

TuningArchiveResourceKind

Module

Gets the Python module of the tuning (its `m` attribute, or the module of module tuning), if it has one

```
public string? Module { get; }
```

Property Value

string

Name

Gets the name of the resource (its tuning name, SimData instance name, or the path of a Python module), if it has one

```
public string? Name { get; }
```

Property Value

string

Pack

Gets the code of the pack the resource came from (`BG` for the base game)

```
public string Pack { get; }
```

Property Value

string

Type

Gets the type of the resource

```
public ResourceType Type { get; }
```

Property Value

ResourceType

Methods

HasSameContentAs(TuningArchiveEntry)

Gets whether this version and the specified `other` have exactly the same content

```
public bool HasSameContentAs(TuningArchiveEntry other)
```

Parameters

`other` TuningArchiveEntry

Returns

bool

ToString()

Returns a string that represents the current object.

```
public override string ToString()
```

Returns

string

A string that represents the current object.

Enum TuningArchiveExportAttributeOrder

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies the order in which ExportAsync(TuningArchiveSnapshot, string, TuningArchiveExportOptions?, IProgress<TuningArchiveProgress>?, CancellationToken) writes the `n` and `t` attributes of an element that has both

```
public enum TuningArchiveExportAttributeOrder
```

Fields

`NameFirst = 0`

The name (`n`) before the type (`t`)

`TypeFirst = 1`

The type (`t`) before the name (`n`)

Enum TuningArchiveExportExistingFiles

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies what ExportAsync(TuningArchiveSnapshot, string, TuningArchiveExportOptions?, IProgress<TuningArchiveProgress>?, CancellationToken) does when a file it would write already exists

```
public enum TuningArchiveExportExistingFiles
```

Fields

`Overwrite = 0`

Replace the file

`Rename = 1`

Rename the file by appending `(Copy 1)` (or the next free number) to its name, then write the new one

`Skip = 2`

Leave the file as it is and write nothing in its place

Enum TuningArchiveExportNaming

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies how ExportAsync(TuningArchiveSnapshot, string, TuningArchiveExportOptions?, IProgress<TuningArchiveProgress>?, CancellationToken) names the files it writes, following the conventions of Scumbumbo's XML Extractor

```
public enum TuningArchiveExportNaming
```

Fields

`Name = 2`

The name alone (such as `object_drink_Cider.xml`)

`Native = 3`

The game's own naming, without an extension (such as `0x00000000!0x00000000000029c3.0x2451c101`)

`Sims4Studio = 4`

The naming Sims 4 Studio uses (such as `b61de6b4!00000001!000000000001c8c5.object_drink_Cider.object.xml`)

`TypeGroupInstance = 0`

The type, group and instance, for importing into s4pe (such as `S4_E882D22F_00000000_000000000001A2BC.xml`)

`TypeGroupInstanceName = 1`

The type, group and instance followed by the name, which s4pe turns into a `_KEY` name on import (such as `S4_B61DE6B4_00000001_000000000001C8C5_object_drink_Cider.xml`)

Class TuningArchiveExportOptions

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents the choices a creator makes when exporting a snapshot as files, the way Scumbumbo's XML Extractor extracts the game's tuning

```
public sealed class TuningArchiveExportOptions
```

Inheritance

object ← TuningArchiveExportOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

AttributeOrder

Gets/sets the order of the `n` and `t` attributes of elements that have both (the default is NameFirst)

```
public TuningArchiveExportAttributeOrder AttributeOrder { get; set; }
```

Property Value

TuningArchiveExportAttributeOrder

ExistingFiles

Gets/sets what to do when a file to be written already exists (the default is Overwrite)

```
public TuningArchiveExportExistingFiles ExistingFiles { get; set; }
```

Property Value

TuningArchiveExportExistingFiles

IncludeAnimationStateMachines

Gets/sets whether to write the snapshot's animation state machines, in an `AsmJazz` folder in each pack's folder (the default is false)

```
public bool IncludeAnimationStateMachines { get; set; }
```

Property Value

bool

IncludeReferenceComments

Gets/sets whether to add a comment naming the tuning or animation state machine each reference refers to (the default is true)

```
public bool IncludeReferenceComments { get; set; }
```

Property Value

bool

IncludeSimData

Gets/sets whether to write the snapshot's SimData, beside the tuning each belongs to (the default is false)

```
public bool IncludeSimData { get; set; }
```

Property Value

bool

IncludeStringComments

Gets/sets whether to add a comment giving the text of each string key (the default is true)

```
public bool IncludeStringComments { get; set; }
```

Property Value

bool

Naming

Gets/sets how to name the files (the default is TypeGroupInstance)

```
public TuningArchiveExportNaming Naming { get; set; }
```

Property Value

TuningArchiveExportNaming

UseTuningDescriptions

Gets/sets whether to find string keys and references with the snapshot's tuning descriptions when it has them (the default is true); otherwise, or without them, every number that matches a string key or the instance of a tuning is taken to be one, as XML Extractor does

```
public bool UseTuningDescriptions { get; set; }
```

Property Value

bool

WriteFileIndex

Gets/sets whether to write `XML File Index.txt` and `XML Cross Reference.dat`, which Scumbumbo's XML File Finder reads (the default is true)

```
public bool WriteFileIndex { get; set; }
```

Property Value

bool

WriteReferenceFiles

Gets/sets whether to write `Reference - All Strings.txt` and `Reference - All XML Names.txt` (the default is true)

```
public bool WriteReferenceFiles { get; set; }
```

Property Value

bool

Class TuningArchiveExportResult

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents the outcome of exporting a snapshot as files

```
public sealed class TuningArchiveExportResult
```

Inheritance

object ← TuningArchiveExportResult

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Duration

Gets how long the export took

```
public TimeSpan Duration { get; }
```

Property Value

TimeSpan

Files

Gets how many resource files were written

```
public int Files { get; }
```

Property Value

int

PackIdentifiers

Gets the identifier used as the group in file names for each pack, by pack code

```
public IReadOnlyDictionary<string, uint> PackIdentifiers { get; }
```

Property Value

IReadOnlyDictionary<string, uint>

Problems

Gets descriptions of anything the creator should know about the export

```
public IReadOnlyList<string> Problems { get; }
```

Property Value

IReadOnlyList<string>

Renamed

Gets how many existing files were renamed to make way for new ones

```
public int Renamed { get; }
```

Property Value

int

Skipped

Gets how many resource files were not written because a file was already there

```
public int Skipped { get; }
```

Property Value

int

UsedTuningDescriptions

Gets whether string keys and references were found with tuning descriptions (rather than by taking every matching number to be one)

```
public bool UsedTuningDescriptions { get; }
```

Property Value

bool

Class TuningArchiveFile

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents an archive of The Sims 4's tuning across versions of the game, stored in a SQLite database file (🔒)

```
public sealed class TuningArchiveFile : IDisposable
```

Inheritance

object ← TuningArchiveFile

Implements

IDisposable

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Each TuningArchiveSnapshot records the tuning, SimData and strings of a version of the game, as captured from an installation or imported from extracted files. Content is stored once however many snapshots contain it, and each resource's history is kept as ranges of snapshots, so a snapshot adds only what changed.

An archive's members are safe to use from multiple threads. Operations that change the archive are serialized.

An application that only reads an archive another application maintains should open it with OpenReadOnly(string), which never changes the file, rather than Open(string), which upgrades the file to this library's version.

Properties

IsReadOnly

Gets whether the archive was opened for reading only (see OpenReadOnly(string))

```
public bool IsReadOnly { get; }
```

Property Value

bool

Path

Gets the path of the archive's database file

```
public string Path { get; }
```

Property Value

string

VoiceCodes

Gets the codes of the voices for which the game keeps versions of an audio configuration that tuning names without a voice, in the order the archive lists the versions (see GetVoiceVersionInstance(ulong, string))

```
public static IReadOnlyList<string> VoiceCodes { get; }
```

Property Value

IReadOnlyList<string>

Remarks

`f` codes are female voices, `m` codes male voices and `c` codes child voices; the game has no versions for `fb` or `md`.

Methods

Analyze(TuningArchiveSnapshot, TuningArchiveEntry)

Analyzes the specified tuning `entry` with the tuning descriptions best suited to the specified `snapshot`, or returns null if the archive has no tuning descriptions

```
public TuningAnalysis? Analyze(TuningArchiveSnapshot snapshot, TuningArchiveEntry entry)
```

Parameters

`snapshot` TuningArchiveSnapshot

`entry` TuningArchiveEntry

Returns

TuningAnalysis

CaptureAsync(GameInstallation, TuningArchiveCaptureOptions?, IProgress<TuningArchiveProgress>?, CancellationToken)

Captures a snapshot of the tuning, SimData, animation state machines, strings and media of the specified `installation` (💤)

```
public Task<TuningArchiveCaptureResult> CaptureAsync(GameInstallation installation,
TuningArchiveCaptureOptions? options = null, IProgress<TuningArchiveProgress>? progress =
null, CancellationToken cancellationToken = default)
```

Parameters

`installation` GameInstallation

`options` TuningArchiveCaptureOptions

`progress` IProgress<TuningArchiveProgress>

`cancellationToken` CancellationToken

Returns

Task<TuningArchiveCaptureResult>

Remarks

Media are the images, sounds and videos to which the captured tuning and SimData refer (with the audio the sounds use), taken from the client packages of the packs with the same rules as tuning; which kinds are captured is up to the `options`. Strings are merged from every string table of every pack the way the game merges them: a string in a package of higher priority replaces the same string in a package of lower priority (packages of the same priority in pack order, the base game first), so a pack's `Delta` strings add to and correct its installed strings rather than replacing them. Within a pack, tuning, SimData and animation state machines are taken from the pack's `Delta` packages before its installed packages, because the `Delta` folder patches the installed one: its resources replace theirs and its delete records retire theirs, whatever priorities the two folders' configurations assign. The game does not load the patches of packs that are not installed, so their tuning is not captured; only the names of the tunings they patch are recorded (see GetUninstalledTunings(TuningArchiveSnapshot)), so that what refers to them can still be explained.

Exceptions

InvalidOperationException

No game version was specified and the installation's version could not be found

NotSupportedException

The archive was opened for reading only

Compare(TuningArchiveSnapshot, TuningArchiveSnapshot, TuningArchiveResourceKind, bool)

Gets the resources of the specified `kind` that differ between the `older` and `newer` snapshots, and, if `includeUnchanged` is true, those that do not (as Unchanged)

```
public IReadOnlyList<TuningArchiveChange> Compare(TuningArchiveSnapshot older,
TuningArchiveSnapshot newer, TuningArchiveResourceKind kind =
TuningArchiveResourceKind.Tuning, bool includeUnchanged = false)
```

Parameters

`older` TuningArchiveSnapshot

`newer` TuningArchiveSnapshot

`kind` TuningArchiveResourceKind

`includeUnchanged` bool

Returns

IReadOnlyList<TuningArchiveChange>

CompareStrings(TuningArchiveSnapshot, TuningArchiveSnapshot, bool)

Gets the strings that differ between the `older` and `newer` snapshots, and, if `includeUnchanged` is true, those that do not (as Unchanged), in the `newer` snapshot's locale

```
public IReadOnlyList<TuningArchiveStringChange> CompareStrings(TuningArchiveSnapshot older,
TuningArchiveSnapshot newer, bool includeUnchanged = false)
```

Parameters

`older` TuningArchiveSnapshot

`newer` TuningArchiveSnapshot

`includeUnchanged` bool

Returns

IReadOnlyList<TuningArchiveStringChange>

Remarks

A string whose text is the same in both snapshots is reported as Moved only when both snapshots were captured from the game, because an extracted strings file does not say which pack each string comes from.

DeleteSnapshotAsync(TuningArchiveSnapshot, CancellationToken)

Deletes the specified `snapshot` and any content no other snapshot shares (💤)

```
public Task DeleteSnapshotAsync(TuningArchiveSnapshot snapshot, CancellationToken
cancellationToken = default)
```

Parameters

`snapshot` TuningArchiveSnapshot

`cancellationToken` CancellationToken

Returns

Task

Exceptions

NotSupportedException

The archive was opened for reading only

DeleteTuningDescriptionsAsync(TuningArchiveDescriptions, CancellationToken)

Deletes the specified set of tuning `descriptions` and the references indexed with it (💤)

```
public Task DeleteTuningDescriptionsAsync(TuningArchiveDescriptions descriptions,
CancellationToken cancellationToken = default)
```

Parameters

`descriptions` TuningArchiveDescriptions

`cancellationToken` CancellationToken

Returns

Task

Exceptions

NotSupportedException

The archive was opened for reading only

Dispose()

Releases the archive's resources

```
public void Dispose()
```

ExportAsync(TuningArchiveSnapshot, string, TuningArchiveExportOptions?, IProgress<TuningArchiveProgress>?, CancellationToken)

Exports the tuning of the specified `snapshot` (and, optionally, its animation state machines and SimData) as files in the specified `folder`, laid out and named the way Scumbumbo's XML Extractor extracts the game's tuning (💤)

```
public Task<TuningArchiveExportResult> ExportAsync(TuningArchiveSnapshot snapshot, string
folder, TuningArchiveExportOptions? options = null, IProgress<TuningArchiveProgress>?
progress = null, CancellationToken cancellationToken = default)
```

Parameters

`snapshot` TuningArchiveSnapshot

`folder` string

`options` TuningArchiveExportOptions

`progress` IProgress<TuningArchiveProgress>

`cancellationToken` CancellationToken

Returns

Task<TuningArchiveExportResult>

Remarks

Each resource is written to a folder for its pack (such as `EP01`) and, in it, a folder for its tuning type (such as `buff`) or `AsmJazz`, as indented UTF-8 XML with a byte order mark and CRLF line endings. SimData is written beside the tuning with its instance, so that both can be chosen together for Sims 4 Studio's batch import, or to a `SimData` folder in its pack's folder if there is no such tuning. The group in file names is the pack's identifier, as XML Extractor writes it: taken from the snapshot's game scripts when it has them, otherwise from the pack stamp of the pack's SimData, otherwise from XML Extractor's own table of packs.

A tuning that more than one pack contains is written once, to the folder of the pack the snapshot attributes it to; XML Extractor writes it to the folder of each pack. The archive does not keep the order of the attributes the game ships, so elements' attributes are in canonical order (see TuningXml), with the `n` and `t` attributes ordered as AttributeOrder specifies. References to tuning of packs that were not installed are commented with the names the packs' patches give it, as XML Extractor comments them (see GetUninstalledTunings(TuningArchiveSnapshot)), and those names are in the reference files, but the tuning itself is not exported.

FindTuningDescriptions(TuningArchiveSnapshot)

Gets the set of tuning descriptions best suited to the specified `snapshot`: the one for its game version, otherwise the nearest earlier one, otherwise the nearest later one

```
public TuningArchiveDescriptions? FindTuningDescriptions(TuningArchiveSnapshot snapshot)
```

Parameters

`snapshot` TuningArchiveSnapshot

Returns

TuningArchiveDescriptions

FindTuningDescriptions(TuningArchiveSnapshot, bool)

Gets the set of tuning descriptions for the game version of the specified `snapshot`: when `exactGameVersion` is true, only the one for that game version (or null if the archive has none), otherwise the one best suited to it (see FindTuningDescriptions(TuningArchiveSnapshot))

```
public TuningArchiveDescriptions? FindTuningDescriptions(TuningArchiveSnapshot snapshot,
```

```
bool exactGameVersion)
```

Parameters

`snapshot` TuningArchiveSnapshot

`exactGameVersion` bool

Returns

TuningArchiveDescriptions

GetBrokenMediaReferences(TuningArchiveSnapshot)

Gets the references in the specified `snapshot` to media it captured that are not in it

```
public IReadOnlyList<TuningArchiveReference> GetBrokenMediaReferences(TuningArchiveSnapshot
snapshot)
```

Parameters

`snapshot` TuningArchiveSnapshot

Returns

IReadOnlyList<TuningArchiveReference>

Remarks

Only references to kinds of media the snapshot captured are considered (see HasKind(TuningArchive Snapshot, TuningArchiveResourceKind)), and only tunings whose references have been indexed (see IndexReferencesAsync(IProgress<TuningArchiveProgress>?, CancellationToken)). A reference to an audio configuration that the snapshot has only voices' versions of is not broken (see GetVoiceVersion Instance(ulong, string)).

GetBrokenReferences(TuningArchiveSnapshot)

Gets the references in the specified `snapshot` to tunings that are not in it

```
public IReadOnlyList<TuningArchiveReference> GetBrokenReferences(TuningArchiveSnapshot
snapshot)
```

Parameters

`snapshot` TuningArchiveSnapshot

Returns

IReadOnlyList<TuningArchiveReference>

Remarks

Only references to tuning types the archive has seen are considered, and only tunings whose references have been indexed (see IndexReferencesAsync(IProgress<TuningArchiveProgress>?, CancellationToken)).

GetContent(TuningArchiveEntry)

Gets the content of the specified `entry` as the archive stores it

```
public byte[] GetContent(TuningArchiveEntry entry)
```

Parameters

`entry` TuningArchiveEntry

Returns

byte[]

Remarks

Media (see Image, Sound and Video) are stored as the game stores them; other kinds are stored as UTF-8 text (see GetText(TuningArchiveEntry)).

GetEntries(TuningArchiveSnapshot, TuningArchiveResourceKind)

Gets the resources of the specified `kind` in the specified `snapshot`

```
public IReadOnlyList<TuningArchiveEntry> GetEntries(TuningArchiveSnapshot snapshot,
TuningArchiveResourceKind kind = TuningArchiveResourceKind.Tuning)
```

Parameters

`snapshot` TuningArchiveSnapshot

`kind` TuningArchiveResourceKind

Returns

IReadOnlyList<TuningArchiveEntry>

GetEntriesByInstance(TuningArchiveSnapshot, TuningArchiveResourceKind, ulong)

Gets the resources of the specified `kind` with the specified `instance` in the specified `snapshot`, whatever their type

```
public IReadOnlyList<TuningArchiveEntry> GetEntriesByInstance(TuningArchiveSnapshot
snapshot, TuningArchiveResourceKind kind, ulong instance)
```

Parameters

`snapshot` TuningArchiveSnapshot

`kind` TuningArchiveResourceKind

`instance` ulong

Returns

IReadOnlyList<TuningArchiveEntry>

GetEntry(TuningArchiveSnapshot, TuningArchiveResourceKind, ResourceType, ulong, uint?)

Gets the resource of the specified `kind`, `type` and `instance` in the specified `snapshot`, if it is there

```
public TuningArchiveEntry? GetEntry(TuningArchiveSnapshot snapshot,
TuningArchiveResourceKind kind, ResourceType type, ulong instance, uint? group = null)
```

Parameters

`snapshot` TuningArchiveSnapshot

`kind` TuningArchiveResourceKind

`type` ResourceType

`instance` ulong

`group` uint?

Returns

TuningArchiveEntry

Remarks

When `group` is null, the resource with the lowest group is returned.

GetHistory(TuningArchiveEntry)

Gets the history of the resource of the specified `entry`: each version it has had and the snapshots containing it

```
public IReadOnlyList<TuningArchiveHistoryItem> GetHistory(TuningArchiveEntry entry)
```

Parameters

`entry` TuningArchiveEntry

Returns

IReadOnlyList<TuningArchiveHistoryItem>

GetMediaEntries(TuningArchiveSnapshot, ResourceKey)

Gets the media in the specified `snapshot` to which tuning or SimData refers with the specified `reference`: the medium GetMediaEntry(TuningArchiveSnapshot, ResourceKey) finds, or, for a reference to an audio configuration that has none of its own, the versions of it for each voice the snapshot contains, in the order of VoiceCodes

```
public IReadOnlyList<TuningArchiveEntry> GetMediaEntries(TuningArchiveSnapshot snapshot,
ResourceKey reference)
```

Parameters

`snapshot` TuningArchiveSnapshot

`reference` ResourceKey

Returns

IReadOnlyList<TuningArchiveEntry>

GetMediaEntry(TuningArchiveSnapshot, ResourceKey)

Gets the medium in the specified `snapshot` to which tuning or SimData refers with the specified `reference`, or null if the snapshot does not contain it

```
public TuningArchiveEntry? GetMediaEntry(TuningArchiveSnapshot snapshot,
ResourceKey reference)
```

Parameters

`snapshot` TuningArchiveSnapshot

`reference` ResourceKey

Returns

TuningArchiveEntry

Remarks

References are resolved as the game resolves them (see GetMediaKind(ResourceType)); a reference to an audio configuration resolves to the configuration rather than to the audio it uses.

GetMediaKind(ResourceType)

Gets the kind of media to which tuning or SimData refers with a resource key of the specified `type`, or null if the type is not one of media

```
public static TuningArchiveResourceKind? GetMediaKind(ResourceType type)
```

Parameters

`type` ResourceType

Returns

TuningArchiveResourceKind?

Remarks

Tuning refers to images with PNGImage and PNGImage2 keys, which the game resolves to the DSTImage with the same instance, and to sounds with AudioConfigurationReference keys, which it resolves to the AudioConfiguration with the same instance, or, when there is none, to the versions of it for each voice (see GetVoiceVersionInstance(ulong, string)).

GetMediaReferrers(TuningArchiveSnapshot, TuningArchiveEntry)

Gets the tunings in the specified `snapshot` that refer to the medium in the specified `entry`

```
public IReadOnlyList<TuningArchiveReference> GetMediaReferrers(TuningArchiveSnapshot
snapshot, TuningArchiveEntry entry)
```

Parameters

`snapshot` TuningArchiveSnapshot

`entry` TuningArchiveEntry

Returns

IReadOnlyList<TuningArchiveReference>

Remarks

References with any of the resource types tuning uses for the medium are found (for an image, PNGImage, PNGImage2 and DSTImage), and, for a voice's version of an audio configuration, references to the configuration it is a version of (see GetVoiceVersionSource(TuningArchiveSnapshot, TuningArchiveEntry)). Only tunings whose references have been indexed (see IndexReferencesAsync(IProgress<TuningArchiveProgress>?, CancellationToken)) are found.

GetMediaResourceTypes(ResourceType)

Gets the resource types to which the game resolves a reference to media with the specified `type`, in the order it tries them (see GetMediaKind(ResourceType)), or only the type itself if it is not a type tuning uses for media

```
public static IReadOnlyList<ResourceType> GetMediaResourceTypes(ResourceType type)
```

Parameters

`type` ResourceType

Returns

IReadOnlyList<ResourceType>

GetPythonModule(TuningArchiveSnapshot, string)

Gets the decompiled source of the specified Python `module` (such as `buffs.buff`) in the specified `snapshot`, if the snapshot has the game's scripts

```
public TuningArchiveEntry? GetPythonModule(TuningArchiveSnapshot snapshot, string module)
```

Parameters

`snapshot` TuningArchiveSnapshot

`module` string

Returns

TuningArchiveEntry

GetPythonModuleInstance(string)

Gets the resource instance under which the archive keeps the Python module at the specified `path` (relative to the decompiled scripts, with forward slashes, such as `buffs/buff.py`)

```
public static ulong GetPythonModuleInstance(string path)
```

Parameters

`path` string

Returns

ulong

GetPythonModulePaths(string)

Gets the path of the decompiled source file of the specified Python `module` (such as `buffs.buff`), relative to the decompiled scripts

```
public static IReadOnlyList<string> GetPythonModulePaths(string module)
```

Parameters

`module` string

Returns

IReadOnlyList<string>

GetReferenceIndexStatus()

Gets how many tunings have had their references indexed, and how many tunings there are

```
public (int indexed, int total) GetReferenceIndexStatus()
```

Returns

(int indexed, int total)

GetReferrers(TuningArchiveSnapshot, ResourceType, ulong)

Gets the tunings in the specified `snapshot` that refer to the resource with the specified `type` and `instance`

```
public IReadOnlyList<TuningArchiveReference> GetReferrers(TuningArchiveSnapshot snapshot,
ResourceType type, ulong instance)
```

Parameters

`snapshot` TuningArchiveSnapshot

`type` ResourceType

`instance` ulong

Returns

IReadOnlyList<TuningArchiveReference>

Remarks

Tunings refer to other tunings and to non-tuning resources (such as object definitions) by tuning reference, and to animation state machines and media by resource key (see GetMediaReferrers(TuningArchiveSnapshot, TuningArchiveEntry) to find a medium's referrers by any of the types tuning uses for it). Only tunings whose references have been indexed (see IndexReferencesAsync(IProgress<TuningArchiveProgress>?, CancellationToken)) are found.

GetSnapshots()

Gets the snapshots in the archive, in order of game version (and then of when they were captured)

```
public IReadOnlyList<TuningArchiveSnapshot> GetSnapshots()
```

Returns

IReadOnlyList<TuningArchiveSnapshot>

GetSoundSamples(TuningArchiveSnapshot, TuningArchiveEntry)

Gets the audio that the audio configuration in the specified `entry` uses (its samples, and those of the configurations it inherits from), as the specified `snapshot` contains them

```
public IReadOnlyList<TuningArchiveEntry> GetSoundSamples(TuningArchiveSnapshot snapshot,
TuningArchiveEntry entry)
```

Parameters

`snapshot` TuningArchiveSnapshot

`entry` TuningArchiveEntry

Returns

IReadOnlyList<TuningArchiveEntry>

GetStringReferrers(TuningArchiveSnapshot, IEnumerable<uint>)

Gets the tunings in the specified `snapshot` that use any of the strings with the specified `keys`

```
public IReadOnlyList<TuningArchiveReference> GetStringReferrers(TuningArchiveSnapshot
snapshot, IEnumerable<uint> keys)
```

Parameters

`snapshot` TuningArchiveSnapshot

`keys` IEnumerable<uint>

Returns

IReadOnlyList<TuningArchiveReference>

Remarks

Each reference's TargetType is StringTable and its TargetInstance is the key of the string used; a tuning using several of the strings appears once for each. Only tunings whose references have been indexed (see IndexReferencesAsync(IProgress<TuningArchiveProgress>?, CancellationToken)) are found.

GetStrings(TuningArchiveSnapshot)

Gets the strings of the specified `snapshot`, by key, in its Locale

```
public IReadOnlyDictionary<uint, string> GetStrings(TuningArchiveSnapshot snapshot)
```

Parameters

`snapshot` TuningArchiveSnapshot

Returns

IReadOnlyDictionary<uint, string>

GetText(TuningArchiveEntry)

Gets the content of the specified `entry`, as canonical text (see TuningXml) or Python source

```
public string GetText(TuningArchiveEntry entry)
```

Parameters

`entry` TuningArchiveEntry

Returns

string

GetTuningDescriptionCache(TuningArchiveDescriptions)

Gets the cache the archive keeps for the specified set of tuning `descriptions`, so that repeated analyses are fast

```
public TuningDescriptionCache GetTuningDescriptionCache(TuningArchiveDescriptions
descriptions)
```

Parameters

`descriptions` TuningArchiveDescriptions

Returns

TuningDescriptionCache

GetTuningDescriptions()

Gets the sets of tuning descriptions imported into the archive, in order of game version

```
public IReadOnlyList<TuningArchiveDescriptions> GetTuningDescriptions()
```

Returns

IReadOnlyList <TuningArchiveDescriptions>

GetTuningTypeNames()

Gets the names of the tuning types the archive has seen, by resource type

```
public IReadOnlyDictionary<ResourceType, string> GetTuningTypeNames()
```

Returns

IReadOnlyDictionary <ResourceType, string >

GetUninstalledTunings(TuningArchiveSnapshot)

Gets the tunings of packs that were not installed when the specified `snapshot` was captured, known only by name from the packs' patches in the game's `Delta` folder

```
public IReadOnlyList<TuningArchiveUninstalledTuning>
GetUninstalledTunings(TuningArchiveSnapshot snapshot)
```

Parameters

`snapshot` TuningArchiveSnapshot

Returns

IReadOnlyList <TuningArchiveUninstalledTuning>

Remarks

None of them is in the snapshot. Only snapshots captured from an installation have them.

GetVoiceVersionInstance(ulong, string)

Gets the instance of the version for the voice with the specified `voiceCode` (see VoiceCodes) of the audio configuration that tuning names with the specified `instance`

```
public static ulong GetVoiceVersionInstance(ulong instance, string voiceCode)
```

Parameters

`instance` ulong

`voiceCode` string

Returns

ulong

Remarks

Tuning names some sounds sims sing or speak (`vocal_clip` in recipes, for example) by the FNV-1 64 hash of a file name that has no audio configuration of its own, such as `vo_karaoke_solojazz2_tune_67sec`. The game has a configuration for each voice instead, hashed from the file name followed by an underscore and the voice's code (`vo_karaoke_solojazz2_tune_67sec_fa`). FNV-1 has no finalization, so the version's instance is found by continuing the hash from the instance tuning names, without the file name.

GetVoiceVersionSource(TuningArchiveSnapshot, TuningArchiveEntry)

Gets the instance by which tuning names the audio configuration of which the specified `entry` is a voice's version, with that voice's code, or null if it is not a voice's version (see GetVoiceVersionInstance(ulong, string))

```
public (ulong instance, string voiceCode)? GetVoiceVersionSource(TuningArchiveSnapshot
snapshot, TuningArchiveEntry entry)
```

Parameters

`snapshot` TuningArchiveSnapshot

`entry` TuningArchiveEntry

Returns

(ulong instance, string voiceCode)?

Remarks

A configuration is taken to be a voice's version when tuning in the specified `snapshot` refers to the configuration it would be a version of (only tunings whose references have been indexed are found; see IndexReferencesAsync(IProgress<TuningArchiveProgress>?, CancellationToken)), or another voice's version of that configuration is in the snapshot. Codes that share their first letter differ only in the last character hashed, so working back with the wrong one of them leads to a configuration with versions of its own in the snapshot (this one, and the version for another voice of the same group); of the configurations it could be a version of, the one tuning refers to is preferred, then the one with the most versions in the snapshot.

GetVoicelessInstance(ulong, string)

Gets the instance of the audio configuration whose version for the voice with the specified `voiceCode` (see VoiceCodes) would have the specified `voiceVersionInstance`: the inverse of GetVoiceVersionInstance(ulong, string)

```
public static ulong GetVoicelessInstance(ulong voiceVersionInstance, string voiceCode)
```

Parameters

`voiceVersionInstance` ulong

`voiceCode` string

Returns

ulong

Remarks

Any instance has a voiceless instance for every code; to tell which configuration an entry is really a version of, use GetVoiceVersionSource(TuningArchiveSnapshot, TuningArchiveEntry).

HasKind(TuningArchiveSnapshot, TuningArchiveResourceKind)

Gets whether the specified `snapshot` contains resources of the specified `kind`

```
public bool HasKind(TuningArchiveSnapshot snapshot, TuningArchiveResourceKind kind)
```

Parameters

`snapshot` TuningArchiveSnapshot

`kind` TuningArchiveResourceKind

Returns

bool

ImportFolderAsync(string, string, TuningArchiveImportOptions?, IProgress<TuningArchiveProgress>?, CancellationToken)

Imports a folder of tuning files extracted by another tool (such as Scumbumbo's XML Extractor or Sims 4 Studio) as a snapshot of the specified `gameVersion` (💤)

```
public Task<TuningArchiveCaptureResult> ImportFolderAsync(string folder, string gameVersion,
TuningArchiveImportOptions? options = null, IProgress<TuningArchiveProgress>? progress =
```

```
null, CancellationToken cancellationToken = default)
```

Parameters

`folder` string

`gameVersion` string

`options` TuningArchiveImportOptions

`progress` IProgress<TuningArchiveProgress>

`cancellationToken` CancellationToken

Returns

Task<TuningArchiveCaptureResult>

Remarks

Every `.xml` file under the folder whose root is an instance (`I`) or module (`M`) tuning element is imported as tuning, every one whose root is `SimData` as SimData, and every other one whose file name has a resource key of type AnimationStateMachine (as Scumbumbo's XML Extractor names the files in its `AsmJazz` folders) as an animation state machine. A tuning's type is derived from its `i` attribute the way the game derives it, falling back to a resource key in its file name, and its group is always 0 (the game identifies tuning by type and instance); comments are discarded. A file is attributed to the pack named by a folder in its path (such as `EP01`), or to DefaultPack.

A file named `Reference - All Strings.txt` in the folder, as Scumbumbo's XML Extractor writes it, is imported as the snapshot's strings.

Exceptions

NotSupportedException

> The archive was opened for reading only

ImportPythonAsync(TuningArchiveSnapshot, string, IProgress<TuningArchiveProgress>?, CancellationToken)

Imports the game's decompiled Python scripts, as Motherlode writes them, into the specified `snapshot`, replacing any it already has (💤)

```
public Task<TuningArchiveCaptureResult> ImportPythonAsync(TuningArchiveSnapshot snapshot,
string folder, IProgress<TuningArchiveProgress>? progress = null, CancellationToken
cancellationToken = default)
```

Parameters

`snapshot` TuningArchiveSnapshot

`folder` string

`progress` IProgress<TuningArchiveProgress>

`cancellationToken` CancellationToken

Returns

Task<TuningArchiveCaptureResult>

Remarks

`folder` is Motherlode's output folder (containing `scripts` and `motherlode-report.json`) or its `scripts` folder. Every `.py` file under `scripts` is imported, keyed by its path relative to it; Motherlode's report, if present, is imported with them, so that the functions it could not verify can be shown.

Exceptions

NotSupportedException

The archive was opened for reading only

ImportTuningDescriptionsAsync(string, string?, CancellationToken)

Imports the ZIP archive of tuning descriptions at the specified `path`, as Maxis distributes them, replacing any set already imported for the same game version (💤)

```
public Task<TuningArchiveDescriptions> ImportTuningDescriptionsAsync(string path, string?
```

```
gameVersion = null, CancellationToken cancellationToken = default)
```

Parameters

`path` string

`gameVersion` string

`cancellationToken` CancellationToken

Returns

Task<TuningArchiveDescriptions>

Remarks

When `gameVersion` is null, it is taken from the name of the file (such as `1.128.90.1030-TDESC.zip`).

Exceptions

InvalidOperationException

No game version was specified and the file's name does not contain one

NotSupportedException

The archive was opened for reading only

IndexReferencesAsync(IProgress<TuningArchiveProgress>?, CancellationToken)

Indexes the references in every tuning that has not yet been indexed with the tuning descriptions best suited to it, returning how many tunings were indexed (💤)

```
public Task<int> IndexReferencesAsync(IProgress<TuningArchiveProgress>? progress = null,
CancellationToken cancellationToken = default)
```

Parameters

`progress` IProgress<TuningArchiveProgress>

`cancellationToken` CancellationToken

Returns

Task<int>

Remarks

The tuning descriptions best suited to a tuning are those best suited to the earliest snapshot containing it (see FindTuningDescriptions(TuningArchiveSnapshot)). Indexing is needed after capturing or importing a snapshot, after importing tuning descriptions, and after opening the archive with a version of this library that indexes differently (which discards the old index); tunings already indexed with the right descriptions are skipped.

Exceptions

NotSupportedException

The archive was opened for reading only

Open(string)

Opens the archive at the specified `path`, creating it if it does not exist (🗑)

```
public static TuningArchiveFile Open(string path)
```

Parameters

`path` string

Returns

TuningArchiveFile

Remarks

An archive created by an earlier version of this library is upgraded, and its reference index is discarded if an earlier version built it (see IndexReferencesAsync(IProgress<TuningArchiveProgress>?, CancellationToken)).

Exceptions

InvalidDataException

The file is an archive created by a newer version of this library

OpenReadOnly(string)

Opens the existing archive at the specified `path` for reading only, without changing the file (🗑)

```
public static TuningArchiveFile OpenReadOnly(string path)
```

Parameters

`path` string

Returns

TuningArchiveFile

Remarks

The archive can be read while another application has it open with Open(string), and sees what that application writes as it is written. Members that would change the archive throw NotSupportedException.

If the archive's reference index was built by a different version of this library, it is left as it is and treated as empty: members that use it find no references, and GetReferenceIndexStatus() reports no tunings indexed.

Exceptions

FileNotFoundException

There is no file at `path`

InvalidDataException

The file is not an archive, or is an archive created by a newer version of this library, or by an earlier version and not yet upgraded by opening it with Open(string)

OpenTuningDescriptions(TuningArchiveDescriptions)

Opens the specified set of tuning `descriptions`; the archive owns the set, so it must not be disposed

```
public TuningDescriptionSet OpenTuningDescriptions(TuningArchiveDescriptions descriptions)
```

Parameters

`descriptions` TuningArchiveDescriptions

Returns

TuningDescriptionSet

UpdateSnapshotAsync(TuningArchiveSnapshot, string?, string?, CancellationToken)

Changes the label and notes of the specified `snapshot`, returning the updated snapshot (💤)

```
public Task<TuningArchiveSnapshot> UpdateSnapshotAsync(TuningArchiveSnapshot snapshot,
string? label, string? notes, CancellationToken cancellationToken = default)
```

Parameters

`snapshot` TuningArchiveSnapshot

`label` string

`notes` string

`cancellationToken` CancellationToken

Returns

Task<TuningArchiveSnapshot>

Exceptions

NotSupportedException

The archive was opened for reading only

Class TuningArchiveHistoryItem

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a version of a resource and the snapshots that contain it

```
public sealed record TuningArchiveHistoryItem : IEquatable<TuningArchiveHistoryItem>
```

Inheritance

object ← TuningArchiveHistoryItem

Implements

IEquatable<TuningArchiveHistoryItem>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningArchiveHistoryItem(TuningArchiveEntry, IReadOnlyList<TuningArchiveSnapshot>)

Represents a version of a resource and the snapshots that contain it

```
public TuningArchiveHistoryItem(TuningArchiveEntry Entry,
IReadOnlyList<TuningArchiveSnapshot> Snapshots)
```

Parameters

`Entry` TuningArchiveEntry

The version of the resource

`Snapshots` IReadOnlyList<TuningArchiveSnapshot>

The snapshots containing this version, in order

Properties

Entry

The version of the resource

```
public TuningArchiveEntry Entry { get; init; }
```

Property Value

TuningArchiveEntry

Snapshots

The snapshots containing this version, in order

```
public IReadOnlyList<TuningArchiveSnapshot> Snapshots { get; init; }
```

Property Value

IReadOnlyList <TuningArchiveSnapshot>

Class TuningArchiveImportOptions

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents the choices a creator makes when importing a folder of tuning files extracted by another tool as a snapshot

```
public sealed class TuningArchiveImportOptions
```

Inheritance

object ← TuningArchiveImportOptions

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

DefaultPack

Gets/sets the code of the pack to attribute files to when their path does not name one (the default is `BG`)

```
public string DefaultPack { get; set; }
```

Property Value

string

Label

Gets/sets a label for the snapshot

```
public string? Label { get; set; }
```

Property Value

string

Locale

Gets/sets the locale of the strings in the folder's string reference file, if it has one (the default is `ENG_US`)

```
public string Locale { get; set; }
```

Property Value

string

Notes

Gets/sets notes to record with the snapshot

```
public string? Notes { get; set; }
```

Property Value

string

Struct TuningArchiveProgress

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents the progress of a long-running TuningArchiveFile operation

```
public readonly record struct TuningArchiveProgress : IEquatable<TuningArchiveProgress>
```

Implements
IEquatable<TuningArchiveProgress>

Inherited Members
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Constructors

TuningArchiveProgress(string, int, int)

Represents the progress of a long-running TuningArchiveFile operation

```
public TuningArchiveProgress(string Stage, int Completed, int Total)
```

Parameters

`Stage` string

A description of what the operation is doing

`Completed` int

How many units of work the stage has completed

`Total` int

How many units of work the stage has, or 0 if it is not known

Properties

Completed

How many units of work the stage has completed

```
public int Completed { get; init; }
```

Property Value

int

Stage

A description of what the operation is doing

```
public string Stage { get; init; }
```

Property Value

string

Total

How many units of work the stage has, or 0 if it is not known

```
public int Total { get; init; }
```

Property Value

int

Class TuningArchiveReference

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a reference from a tuning to another tuning or resource

```
public sealed record TuningArchiveReference : IEquatable<TuningArchiveReference>
```

Inheritance

object ← TuningArchiveReference

Implements

IEquatable<TuningArchiveReference>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningArchiveReference(TuningArchiveEntry, ResourceType, ulong)

Represents a reference from a tuning to another tuning or resource

```
public TuningArchiveReference(TuningArchiveEntry Source, ResourceType TargetType,
ulong TargetInstance)
```

Parameters

`Source` TuningArchiveEntry

The tuning containing the reference

`TargetType` ResourceType

The type of the resource referred to

`TargetInstance` ulong

The instance of the resource referred to

Properties

Source

The tuning containing the reference

```
public TuningArchiveEntry Source { get; init; }
```

Property Value

TuningArchiveEntry

TargetInstance

The instance of the resource referred to

```
public ulong TargetInstance { get; init; }
```

Property Value

ulong

TargetType

The type of the resource referred to

```
public ResourceType TargetType { get; init; }
```

Property Value

ResourceType

Enum TuningArchiveResourceKind

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies the kind of a resource in a TuningArchiveFile

```
public enum TuningArchiveResourceKind
```

Fields

`AnimationStateMachine = 3`

An animation state machine (also known as an ASM or JAZZ)

`Image = 4`

An image to which tuning or SimData refers (such as an icon), stored as the game stores it (usually a DSTImage)

`PythonModule = 2`

A Python module of the game's scripts, as decompiled source

`SimData = 1`

A SimData resource, stored as XML in the format Sims 4 Studio uses

`Sound = 5`

A sound to which tuning or SimData refers: an audio configuration (AudioConfiguration), or audio one uses (AudioVocals or AudioEffects), stored as the game stores it

`Tuning = 0`

A tuning document (instance or module tuning)

`Video = 6`

A video to which tuning or SimData refers (AVI), stored as the game stores it

Class TuningArchiveSnapshot

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents the state of the game's tuning at a point in time, as captured in a TuningArchiveFile

```
public sealed class TuningArchiveSnapshot
```

Inheritance

object ← TuningArchiveSnapshot

Inherited Members

object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

Properties

CapturedAt

Gets when the snapshot was captured

```
public DateTimeOffset CapturedAt { get; }
```

Property Value

DateTimeOffset

DisplayName

Gets the name to display for the snapshot: its label if it has one, otherwise its game version

```
public string DisplayName { get; }
```

Property Value

string

GameVersion

Gets the version of the game the snapshot represents

```
public string GameVersion { get; }
```

Property Value

string

Id

Gets the identifier of the snapshot within its archive

```
public long Id { get; }
```

Property Value

long

Kinds

Gets the kinds of resources the snapshot contains

```
public IReadOnlySet<TuningArchiveResourceKind> Kinds { get; }
```

Property Value

IReadOnlySet<TuningArchiveResourceKind>

Label

Gets the label the creator gave the snapshot, if any

```
public string? Label { get; }
```

Property Value

string

Locale

Gets the locale of the snapshot's strings, if it has any

```
public string? Locale { get; }
```

Property Value

string

Notes

Gets the notes the creator wrote about the snapshot, if any

```
public string? Notes { get; }
```

Property Value

string

Packs

Gets the packs that were present when the snapshot was taken

```
public IReadOnlyList<TuningArchiveSnapshotPack> Packs { get; }
```

Property Value

IReadOnlyList<TuningArchiveSnapshotPack>

Source

Gets where the contents of the snapshot came from

```
public TuningArchiveSnapshotSource Source { get; }
```

Property Value

TuningArchiveSnapshotSource

SourcePath

Gets the path of the installation or folder the snapshot came from, if it is known

```
public string? SourcePath { get; }
```

Property Value

string

Methods

Equals(object?)

Determines whether the specified object is equal to the current object.

```
public override bool Equals(object? obj)
```

Parameters

`obj` object

The object to compare with the current object.

Returns

bool

true if the specified object is equal to the current object; otherwise, false.

GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

Returns

int

A hash code for the current object.

IsCaptureOf(GameInstallation)

Gets whether the snapshot was captured from an installation of the same game version as the specified `installation`, with the same packs at the same versions

```
public bool IsCaptureOf(GameInstallation installation)
```

Parameters

`installation` GameInstallation

Returns

bool

Remarks

The kinds of resources the snapshot contains are not considered (see Kinds).

ToString()

Returns a string that represents the current object.

```
public override string ToString()
```

Returns

string

A string that represents the current object.

Class TuningArchiveSnapshotPack

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a pack that was present when a TuningArchiveSnapshot was taken

```
public sealed record TuningArchiveSnapshotPack : IEquatable<TuningArchiveSnapshotPack>
```

Inheritance

object ← TuningArchiveSnapshotPack

Implements

IEquatable<TuningArchiveSnapshotPack>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningArchiveSnapshotPack(string, string?)

Represents a pack that was present when a TuningArchiveSnapshot was taken

```
public TuningArchiveSnapshotPack(string Code, string? Version)
```

Parameters

`Code` string

The code of the pack (`BG` for the base game)

`Version` string

The version of the pack, if it was known

Properties

Code

The code of the pack (`BG` for the base game)

```
public string Code { get; init; }
```

Property Value

string

Version

The version of the pack, if it was known

```
public string? Version { get; init; }
```

Property Value

string

Enum TuningArchiveSnapshotSource

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies where the contents of a TuningArchiveSnapshot came from

```
public enum TuningArchiveSnapshotSource
```

Fields

`GameInstallation = 0`

The snapshot was captured from an installation of the game

`TuningFolder = 1`

The snapshot was imported from a folder of tuning files extracted by another tool

Class TuningArchiveStringChange

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents how a string differs, if at all, between two snapshots

```
public sealed record TuningArchiveStringChange : IEquatable<TuningArchiveStringChange>
```

Inheritance

object ← TuningArchiveStringChange

Implements

IEquatable<TuningArchiveStringChange>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningArchiveStringChange(TuningArchiveChangeKind, uint, string?, string?)

Represents how a string differs, if at all, between two snapshots

```
public TuningArchiveStringChange(TuningArchiveChangeKind Kind, uint Key, string? OldText,
string? NewText)
```

Parameters

`Kind` TuningArchiveChangeKind

How the string differs (Moved means only the pack it came from changed)

`Key` uint

The key of the string

`OldText` string

The text of the string in the older snapshot, if it was there

`NewText` string

The text of the string in the newer snapshot, if it is there

Properties

Key

The key of the string

```
public uint Key { get; init; }
```

Property Value

uint

Kind

How the string differs (Moved means only the pack it came from changed)

```
public TuningArchiveChangeKind Kind { get; init; }
```

Property Value

TuningArchiveChangeKind

NewText

The text of the string in the newer snapshot, if it is there

```
public string? NewText { get; init; }
```

Property Value

string

OldText

The text of the string in the older snapshot, if it was there

```
public string? OldText { get; init; }
```

Property Value

string

Class TuningArchiveUninstalledTuning

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a tuning of a pack that was not installed when a TuningArchiveSnapshot was captured, known only from the pack's patches in the game's `Delta` folder

```
public sealed record TuningArchiveUninstalledTuning :
IEquatable<TuningArchiveUninstalledTuning>
```

Inheritance

object ← TuningArchiveUninstalledTuning

Implements

IEquatable<TuningArchiveUninstalledTuning>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The game does not load a pack's patches unless the pack is installed, so such tuning is not in the snapshot; only its name is kept, so that what refers to it can still be explained.

Constructors

TuningArchiveUninstalledTuning(ResourceType, ulong, string, string?, string)

Represents a tuning of a pack that was not installed when a TuningArchiveSnapshot was captured, known only from the pack's patches in the game's `Delta` folder

```
public TuningArchiveUninstalledTuning(ResourceType Type, ulong Instance, string Name,
string? Class, string Pack)
```

Parameters

`Type` ResourceType

The type of the tuning

`Instance` ulong

The instance of the tuning

`Name` string

The name of the tuning

`Class` string

The class of the tuning, if it is an instance tuning

`Pack` string

The code of the pack the tuning belongs to

Remarks

The game does not load a pack's patches unless the pack is installed, so such tuning is not in the snapshot; only its name is kept, so that what refers to it can still be explained.

Properties

Class

The class of the tuning, if it is an instance tuning

```
public string? Class { get; init; }
```

Property Value

string

Instance

The instance of the tuning

```
public ulong Instance { get; init; }
```

Property Value

ulong

Name

The name of the tuning

```
public string Name { get; init; }
```

Property Value

string

Pack

The code of the pack the tuning belongs to

```
public string Pack { get; init; }
```

Property Value

string

Type

The type of the tuning

```
public ResourceType Type { get; init; }
```

Property Value

ResourceType

Class TuningCommenter

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Adds to tuning the comments XML Extractor adds, as a snapshot knows them: the text of each string after its key, and the class and name of the tuning or animation state machine each reference refers to

```
public sealed class TuningCommenter
```

Inheritance

object ← TuningCommenter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The comments are the ones ExportAsync(TuningArchiveSnapshot, string, TuningArchiveExportOptions?, IProgress<TuningArchiveProgress>?, CancellationToken) writes: found with the snapshot's tuning descriptions when it has them and they describe the tuning, and otherwise as XML Extractor finds them, as any number that is a string's key or a tuning's instance. References to tuning of packs that were not installed when the snapshot was captured are commented with the names the packs' patches give it (see GetUninstalledTunings(TuningArchiveSnapshot)). The game ignores comments, so tuning with them loads as it would without them.

Properties

ReferenceComments

Gets whether references are commented with the class and name of what they refer to

```
public bool ReferenceComments { get; }
```

Property Value

bool

StringComments

Gets whether string keys are commented with the text of their strings

```
public bool StringComments { get; }
```

Property Value

bool

Methods

Comment(string)

Gets the specified tuning `text` with comments, or the text as it is if it is not XML

```
public string Comment(string text)
```

Parameters

`text` string

Returns

string

Remarks

Comments already in the text are kept. The result is indented with two spaces, with the line ends the text has, and starts with an XML declaration if the text does.

CommentFragment(string)

Gets the specified fragment of tuning (one element, such as GetFragmentText(XElement) gets) with comments on one line, or the fragment as it is if it is not XML

```
public string CommentFragment(string fragment)
```

Parameters

`fragment` string

Returns

string

Remarks

A fragment has no tuning around it for its tuning descriptions to describe, so its comments are found as XML Extractor finds them.

Create(TuningArchiveFile, TuningArchiveSnapshot, bool, bool)

Creates a commenter with what the specified `snapshot` of the specified `archive` knows

```
public static TuningCommenter Create(TuningArchiveFile archive, TuningArchiveSnapshot
snapshot, bool stringComments = true, bool referenceComments = true)
```

Parameters

`archive` TuningArchiveFile

The archive

`snapshot` TuningArchiveSnapshot

The snapshot whose strings, tuning names and tuning descriptions the comments come from

`stringComments` bool

Whether to comment string keys with the text of their strings

`referenceComments` bool

Whether to comment references with the class and name of what they refer to

Returns

TuningCommenter

Class TuningCoreLibraryDraft

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Drafts a Lot 51 Core Library tuning injector snippet (for Core Library 1.43) that makes changes to tuning without overriding it

```
public static class TuningCoreLibraryDraft
```

Inheritance

object ← TuningCoreLibraryDraft

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The snippet adds items to the lists Core Library's injectors can add to, for the tuning types they cover: object interactions, affordance lists, buff and trait loot, interactions, buffs and commodities, loot actions and random weighted loot, role state buffs, loot and interactions, situation venues and tags, relationship bit loot, interactions and buffs, venue buffs and zone modifiers, and zone modifier loot. Core Library skips items a list already has. Changes to the tests, autonomous tests and global tests of interactions, and to the tests of test set snippets, are made by adding tests where the change only adds new test lists or the same tests to every list, and otherwise by replacing the tests, which is reported as a warning. Any other change, and any removal, is reported as an error and left out.

Methods

Generate(IReadOnlyList<TuningDraftTarget>, string)

Drafts a snippet with the specified `name` (by convention `creator:mod`, which also names the creator and the mod for Core Library's messages) that makes the changes in the specified `targets`

```
public static TuningDraftResult Generate(IReadOnlyList<TuningDraftTarget> targets,
string name)
```

Parameters

`targets` IReadOnlyList<TuningDraftTarget>

`name` string

Returns

TuningDraftResult

Class TuningDescriptionCache

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Keeps the documents of a TuningDescriptionSet that analysis has used, so that analyzing many tunings does not read and parse the same descriptions repeatedly (🔒)

```
public sealed class TuningDescriptionCache
```

Inheritance

object ← TuningDescriptionCache

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningDescriptionCache(TuningDescriptionSet)

Initializes a new cache for the specified `set`

```
public TuningDescriptionCache(TuningDescriptionSet set)
```

Parameters

`set` TuningDescriptionSet

Properties

Set

Gets the set of tuning descriptions

```
public TuningDescriptionSet Set { get; }
```

Property Value

TuningDescriptionSet

Methods

Clear()

Releases the documents the cache keeps

```
public void Clear()
```

Resolve(TunableDescription)

Gets the description the specified `description` stands for: the description of the fragment it refers to if it is a FragmentReferenceTunableDescription, and otherwise itself

```
public TunableDescription Resolve(TunableDescription description)
```

Parameters

`description` TunableDescription

Returns

TunableDescription

Exceptions

InvalidDataException

The set does not have the fragment to which `description` refers

Class TuningDraftDiagnostic

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents what a draft does, or cannot do, about a change to a tunable

```
public sealed record TuningDraftDiagnostic : IEquatable<TuningDraftDiagnostic>
```

Inheritance

object ← TuningDraftDiagnostic

Implements

IEquatable<TuningDraftDiagnostic>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningDraftDiagnostic(TuningDraftSeverity, TuningArchiveEntry, string, string)

Represents what a draft does, or cannot do, about a change to a tunable

```
public TuningDraftDiagnostic(TuningDraftSeverity Severity, TuningArchiveEntry Entry, string
Path, string Message)
```

Parameters

`Severity` TuningDraftSeverity

Whether the draft makes the change

`Entry` TuningArchiveEntry

The tuning that was changed

`Path` string

The path of the tunable that was changed, such as `loot_list[2].stat`

`Message` string

What the draft does, or why it cannot

Properties

Entry

The tuning that was changed

```
public TuningArchiveEntry Entry { get; init; }
```

Property Value

TuningArchiveEntry

Message

What the draft does, or why it cannot

```
public string Message { get; init; }
```

Property Value

string

Path

The path of the tunable that was changed, such as `loot_list[2].stat`

```
public string Path { get; init; }
```

Property Value

string

Severity

Whether the draft makes the change

```
public TuningDraftSeverity Severity { get; init; }
```

Property Value

TuningDraftSeverity

Class TuningDraftResult

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a draft that makes changes to tuning without overriding it: a script or a snippet

```
public sealed record TuningDraftResult : IEquatable<TuningDraftResult>
```

Inheritance

object ← TuningDraftResult

Implements

IEquatable<TuningDraftResult>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningDraftResult(string, IReadOnlyList<TuningDraftDiagnostic>, int)

Represents a draft that makes changes to tuning without overriding it: a script or a snippet

```
public TuningDraftResult(string Source, IReadOnlyList<TuningDraftDiagnostic> Diagnostics,
int Changes)
```

Parameters

`Source` string

The source of the draft (Python, or snippet tuning XML)

`Diagnostics` IReadOnlyList<TuningDraftDiagnostic>

What the draft does about each change, and which changes it cannot make

`Changes` int

The number of changes the draft makes

Properties

Changes

The number of changes the draft makes

```
public int Changes { get; init; }
```

Property Value

int

Diagnostics

What the draft does about each change, and which changes it cannot make

```
public IReadOnlyList<TuningDraftDiagnostic> Diagnostics { get; init; }
```

Property Value

IReadOnlyList <TuningDraftDiagnostic>

Source

The source of the draft (Python, or snippet tuning XML)

```
public string Source { get; init; }
```

Property Value

string

Enum TuningDraftSeverity

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies what a draft can do about a change

```
public enum TuningDraftSeverity
```

Fields

`Error = 2`

The draft cannot make the change, so it leaves it out

`Information = 0`

The draft makes the change

`Warning = 1`

The draft makes the change, with a caveat the creator should know

Class TuningDraftTarget

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a tuning a creator has changed in a draft: the tuning as the archive has it and as the creator wants it

```
public sealed record TuningDraftTarget : IEquatable<TuningDraftTarget>
```

Inheritance

object ← TuningDraftTarget

Implements

IEquatable<TuningDraftTarget>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningDraftTarget(TuningArchiveEntry, TuningAnalysis, TuningAnalysis, bool)

Represents a tuning a creator has changed in a draft: the tuning as the archive has it and as the creator wants it

```
public TuningDraftTarget(TuningArchiveEntry Entry, TuningAnalysis Original, TuningAnalysis
Draft, bool HasSimData = true)
```

Parameters

`Entry` TuningArchiveEntry

The tuning

`Original` TuningAnalysis

The analysis of the tuning as the archive has it

`Draft` TuningAnalysis

The analysis of the tuning as the creator wants it

`HasSimData` bool

Whether the game has SimData for the tuning; when it doesn't, nothing reads the tunables the tuning descriptions export to SimData from SimData, so drafts that can't change SimData can still change them

Properties

Draft

The analysis of the tuning as the creator wants it

```
public TuningAnalysis Draft { get; init; }
```

Property Value

TuningAnalysis

Entry

The tuning

```
public TuningArchiveEntry Entry { get; init; }
```

Property Value

TuningArchiveEntry

HasSimData

Whether the game has SimData for the tuning; when it doesn't, nothing reads the tunables the tuning descriptions export to SimData from SimData, so drafts that can't change SimData can still change them

```
public bool HasSimData { get; init; }
```

Property Value

bool

Original

The analysis of the tuning as the archive has it

```
public TuningAnalysis Original { get; init; }
```

Property Value

TuningAnalysis

Class TuningFolderUpdate

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Finds the overrides of game tuning in every package in a folder and its subfolders, works out what a game version means for each, checks the SimData of the tuning the packages add against the game version's tuning descriptions, and writes updated copies of the packages with the changes that need no judgment from the creator, with a report of everything

```
public sealed class TuningFolderUpdate
```

Inheritance

object ← TuningFolderUpdate

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The changes that need no judgment are merges without conflicts whose base is not uncertain (see Base IsUncertain), the SimData changes and fixes the options allow, and, if the options say so, removing overrides that do nothing. Everything else is left as the package has it and marked in the report as needing the creator.

The packages in the folder are never changed, and updated copies are never written inside it, where the game would load them beside the originals.

Properties

DeclaredBase

Gets the game version the creator said the packages were last updated for, if they said

```
public TuningArchiveSnapshot? DeclaredBase { get; }
```

Property Value

TuningArchiveSnapshot

Folder

Gets the folder

```
public string Folder { get; }
```

Property Value

string

Packages

Gets the packages in the folder that override game tuning, add tuning whose SimData isn't what the tuning descriptions give, or couldn't be read, in order of their paths

```
public IReadOnlyList<TuningFolderUpdatePackage> Packages { get; }
```

Property Value

IReadOnlyList<TuningFolderUpdatePackage>

Target

Gets the snapshot of the game version the packages are being updated for

```
public TuningArchiveSnapshot Target { get; }
```

Property Value

TuningArchiveSnapshot

Methods

AnalyzeAsync(TuningArchiveFile, TuningArchiveSnapshot, string, TuningSimDataDrafter?, TuningArchiveSnapshot?, IProgress<TuningArchiveProgress>?, CancellationToken)

Finds the overrides of game tuning in every package in the specified `folder` and its subfolders and works out what the `target` game version in the specified `archive` means for each (🏃)

```
public static Task<TuningFolderUpdate> AnalyzeAsync(TuningArchiveFile archive,
TuningArchiveSnapshot target, string folder, TuningSimDataDrafter? simData = null,
TuningArchiveSnapshot? declaredBase = null, IProgress<TuningArchiveProgress>? progress =
null, CancellationToken cancellationToken = default)
```

Parameters

`archive` TuningArchiveFile

The archive with the game versions

`target` TuningArchiveSnapshot

The game version to update the packages for

`folder` string

The folder

`simData` TuningSimDataDrafter

A drafter for the `target` snapshot, to check the packages' SimData against what their overrides need and the SimData of the tuning they add against the tuning descriptions, or null not to

`declaredBase` TuningArchiveSnapshot

The game version the creator says the packages were last updated for, to use as the base instead of inferring it, or null to infer it

`progress` IProgress<TuningArchiveProgress>

Receives progress

`cancellationToken` CancellationToken

The token to monitor for cancellation requests

Returns

Task<TuningFolderUpdate>

GetReport(TuningFolderUpdateOptions, IReadOnlyList<TuningFolderUpdateWrite>?)

Gets a report of every override in the folder and the SimData of the tuning the packages add, as comma-separated values with a header row (see GetReportRows(TuningFolderUpdateOptions, IReadOnlyList<TuningFolderUpdateWrite>?))

```
public string GetReport(TuningFolderUpdateOptions options,
IReadOnlyList<TuningFolderUpdateWrite>? written = null)
```

Parameters

`options` TuningFolderUpdateOptions

`written` IReadOnlyList<TuningFolderUpdateWrite>

Returns

string

GetReportRows(TuningFolderUpdateOptions, IReadOnlyList<TuningFolderUpdateWrite>?)

Gets a row for every override in the folder, for the SimData of every tuning the packages add that isn't what the tuning descriptions give, and for every package that couldn't be read, for the specified `options` and, if the updated copies were written, what writing them did

```
public IReadOnlyList<TuningFolderUpdateReportRow> GetReportRows(TuningFolderUpdateOptions
options, IReadOnlyList<TuningFolderUpdateWrite>? written = null)
```

Parameters

`options` TuningFolderUpdateOptions

`written` IReadOnlyList<TuningFolderUpdateWrite>

Returns

IReadOnlyList<TuningFolderUpdateReportRow>

GetTuning(TuningPackageUpdate, TuningFolderUpdateOptions)

Gets the tuning the update writes in place of the specified `update`'s overrides with the specified `options`

```
public static IReadOnlyDictionary<ResourceKey, string?> GetTuning(TuningPackageUpdate
update, TuningFolderUpdateOptions options)
```

Parameters

`update` TuningPackageUpdate

`options` TuningFolderUpdateOptions

Returns

IReadOnlyDictionary<ResourceKey, string>

IsAutomatic(TuningPackageOverride)

Gets whether the update makes the change to the specified `item` on its own: the merge of a mod's changes without conflicts, whose base is not uncertain

```
public static bool IsAutomatic(TuningPackageOverride item)
```

Parameters

`item` TuningPackageOverride

Returns

bool

NeedsCreator(TuningPackageAddedSimData, TuningFolderUpdateOptions)

Gets whether the specified `item` needs the creator's judgment after the update makes the fixes the specified `options` allow

```
public static bool NeedsCreator(TuningPackageAddedSimData item, TuningFolderUpdateOptions
options)
```

Parameters

`item` TuningPackageAddedSimData

`options` TuningFolderUpdateOptions

Returns

bool

NeedsCreator(TuningPackageOverride, TuningFolderUpdateOptions)

Gets whether the specified `item` needs the creator's judgment after the update makes the changes the specified `options` allow

```
public static bool NeedsCreator(TuningPackageOverride item, TuningFolderUpdateOptions
options)
```

Parameters

`item` TuningPackageOverride

`options` TuningFolderUpdateOptions

Returns

bool

Remarks

An override whose declared base disagrees always does, even if it looks up to date: when the creator's declared version is wrong for it, it may be undoing the game's changes since the version it was really made from.

WriteAsync(string, TuningFolderUpdateOptions, bool, IProgress<TuningArchiveProgress>?, CancellationToken)

Writes an updated copy of each package the specified `options` change to the same relative path in the specified `outputFolder`, skipping any whose path there already has a file, and, unless told not to, the report beside them (🏃)

```
public Task<TuningFolderUpdateResult> WriteAsync(string outputFolder,
TuningFolderUpdateOptions options, bool writeReport = true,
IProgress<TuningArchiveProgress>? progress = null, CancellationToken cancellationToken
= default)
```

Parameters

`outputFolder` string

Where to write the updated copies, which must not be in the folder

`options` TuningFolderUpdateOptions

Which changes to make

`writeReport` bool

Whether to write the report (see GetReport(TuningFolderUpdateOptions, IReadOnlyList<TuningFolderUpdateWrite>?)) beside the copies, as `Update report for` the game version, with `.csv`

`progress` IProgress<TuningArchiveProgress>

Receives progress

`cancellationToken` CancellationToken

The token to monitor for cancellation requests

Returns

Task<TuningFolderUpdateResult>

Exceptions

ArgumentException

`outputFolder` is the folder or in it

WriteReportAsync(string, TuningFolderUpdateOptions, IReadOnlyList<TuningFolderUpdateWrite>?, CancellationToken)

Writes the report (see GetReport(TuningFolderUpdateOptions, IReadOnlyList<TuningFolderUpdateWrite>?)) to the specified `path`, as UTF-8 with a byte order mark so that Excel reads it as UTF-8 (🏃)

```
public Task WriteReportAsync(string path, TuningFolderUpdateOptions options,
IReadOnlyList<TuningFolderUpdateWrite>? written = null, CancellationToken cancellationToken
= default)
```

Parameters

`path` string

`options` TuningFolderUpdateOptions

`written` IReadOnlyList<TuningFolderUpdateWrite>

`cancellationToken` CancellationToken

Returns

Task

Class TuningFolderUpdateOptions

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents which changes a folder update makes beyond the merges that need no judgment from the creator

```
public sealed record TuningFolderUpdateOptions : IEquatable<TuningFolderUpdateOptions>
```

Inheritance

object ← TuningFolderUpdateOptions

Implements

IEquatable<TuningFolderUpdateOptions>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningFolderUpdateOptions(bool, TuningPackageSimDataChanges, TuningPackageAddedSimDataFixes, TuningCommenter?)

Represents which changes a folder update makes beyond the merges that need no judgment from the creator

```
public TuningFolderUpdateOptions(bool RemoveOverridesThatDoNothing = false,
TuningPackageSimDataChanges SimDataChanges = TuningPackageSimDataChanges.Replace |
TuningPackageSimDataChanges.Remove, TuningPackageAddedSimDataFixes AddedSimDataFixes =
TuningPackageAddedSimDataFixes.AddMissingColumns, TuningCommenter? Commenter = null)
```

Parameters

`RemoveOverridesThatDoNothing` bool

Whether overrides the same as the game's new version are removed

`SimDataChanges` TuningPackageSimDataChanges

Which changes are made to the SimData of overrides

`AddedSimDataFixes` TuningPackageAddedSimDataFixes

Which fixes are made to the SimData of tuning the packages add

`Commenter` TuningCommenter

Comments the tuning written in place of overrides, if specified

Properties

AddedSimDataFixes

Which fixes are made to the SimData of tuning the packages add

```
public TuningPackageAddedSimDataFixes AddedSimDataFixes { get; init; }
```

Property Value

TuningPackageAddedSimDataFixes

Commenter

Comments the tuning written in place of overrides, if specified

```
public TuningCommenter? Commenter { get; init; }
```

Property Value

TuningCommenter

RemoveOverridesThatDoNothing

Whether overrides the same as the game's new version are removed

```
public bool RemoveOverridesThatDoNothing { get; init; }
```

Property Value

bool

SimDataChanges

Which changes are made to the SimData of overrides

```
public TuningPackageSimDataChanges SimDataChanges { get; init; }
```

Property Value

TuningPackageSimDataChanges

Class TuningFolderUpdatePackage

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a package in a folder being updated

```
public sealed record TuningFolderUpdatePackage : IEquatable<TuningFolderUpdatePackage>
```

Inheritance

object ← TuningFolderUpdatePackage

Implements

IEquatable<TuningFolderUpdatePackage>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningFolderUpdatePackage(string, TuningPackageUpdate?, string?, IReadOnlyList<TuningPackageAddedSimData>)

Represents a package in a folder being updated

```
public TuningFolderUpdatePackage(string RelativePath, TuningPackageUpdate? Update, string?
Problem, IReadOnlyList<TuningPackageAddedSimData> AddedSimData)
```

Parameters

`RelativePath` string

The path of the package within the folder

`Update` TuningPackageUpdate

What the game version means for the package's overrides, if it could be read

`Problem` string

Why the package couldn't be read, if it couldn't

`AddedSimData` IReadOnlyList<TuningPackageAddedSimData>

The SimData of the tuning the package adds that isn't what the tuning descriptions give, if it was checked

Properties

AddedSimData

The SimData of the tuning the package adds that isn't what the tuning descriptions give, if it was checked

```
public IReadOnlyList<TuningPackageAddedSimData> AddedSimData { get; init; }
```

Property Value

IReadOnlyList<TuningPackageAddedSimData>

Problem

Why the package couldn't be read, if it couldn't

```
public string? Problem { get; init; }
```

Property Value

string

RelativePath

The path of the package within the folder

```
public string RelativePath { get; init; }
```

Property Value

string

Update

What the game version means for the package's overrides, if it could be read

```
public TuningPackageUpdate? Update { get; init; }
```

Property Value

TuningPackageUpdate

Class TuningFolderUpdateReportRow

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a row of a folder update's report: an override, the SimData of a tuning a package adds, or a package that couldn't be read

```
public sealed record TuningFolderUpdateReportRow : IEquatable<TuningFolderUpdateReportRow>
```

Inheritance

object ← TuningFolderUpdateReportRow

Implements

IEquatable<TuningFolderUpdateReportRow>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningFolderUpdateReportRow(string, TuningPackageOverride?, TuningPackageAddedSimData?, string, string, string, string, bool, string, string, string, string, int, string, string, string)

Represents a row of a folder update's report: an override, the SimData of a tuning a package adds, or a package that couldn't be read

```
public TuningFolderUpdateReportRow(string Package, TuningPackageOverride? Override,
TuningPackageAddedSimData? AddedSimData, string Tuning, string Type, string Instance, string
Outcome, bool NeedsCreator, string Done, string Base, string BaseSource, string BaseNote,
int Conflicts, string SimData, string Message, string SimDataMessage)
```

Parameters

`Package` string

The path of the package within the folder

`Override` TuningPackageOverride

The override, if the row is for one

`AddedSimData` TuningPackageAddedSimData

The SimData of a tuning the package adds, if the row is for one

`Tuning` string

The name of the tuning

`Type` string

The name of the tuning's type

`Instance` string

The tuning's instance, in decimal

`Outcome` string

What the game version means for the override, for the creator

`NeedsCreator` bool

Whether the override or SimData needs the creator's judgment

`Done` string

What writing the updated copies did to the override or SimData, if they were written

`Base` string

The game version the override was made from

`BaseSource` string

`declared` or `inferred`

`BaseNote` string

Why the base may not be what the override was made from, if it may not be

`Conflicts` int

How many of the mod's changes conflict with the game's

`SimData` string

Whether the override's SimData is what its tuning needs, if it was checked, or what is wrong with the SimData of a tuning the package adds

`Message` string

What the outcome means, or what is wrong with the SimData of a tuning the package adds, for the creator

`SimDataMessage` string

What the SimData state means, if it isn't current, or what the update does to the SimData of a tuning the package adds

Properties

AddedSimData

The SimData of a tuning the package adds, if the row is for one

```
public TuningPackageAddedSimData? AddedSimData { get; init; }
```

Property Value

TuningPackageAddedSimData

Base

The game version the override was made from

```
public string Base { get; init; }
```

Property Value

string

BaseNote

Why the base may not be what the override was made from, if it may not be

```
public string BaseNote { get; init; }
```

Property Value

string

BaseSource

`declared` or `inferred`

```
public string BaseSource { get; init; }
```

Property Value

string

Conflicts

How many of the mod's changes conflict with the game's

```
public int Conflicts { get; init; }
```

Property Value

int

Done

What writing the updated copies did to the override or SimData, if they were written

```
public string Done { get; init; }
```

Property Value

string

Instance

The tuning's instance, in decimal

```
public string Instance { get; init; }
```

Property Value

string

Message

What the outcome means, or what is wrong with the SimData of a tuning the package adds, for the creator

```
public string Message { get; init; }
```

Property Value

string

NeedsCreator

Whether the override or SimData needs the creator's judgment

```
public bool NeedsCreator { get; init; }
```

Property Value

bool

Outcome

What the game version means for the override, for the creator

```
public string Outcome { get; init; }
```

Property Value

string

Override

The override, if the row is for one

```
public TuningPackageOverride? Override { get; init; }
```

Property Value

TuningPackageOverride

Package

The path of the package within the folder

```
public string Package { get; init; }
```

Property Value

string

SimData

Whether the override's SimData is what its tuning needs, if it was checked, or what is wrong with the SimData of a tuning the package adds

```
public string SimData { get; init; }
```

Property Value

string

SimDataMessage

What the SimData state means, if it isn't current, or what the update does to the SimData of a tuning the package adds

```
public string SimDataMessage { get; init; }
```

Property Value

string

Tuning

The name of the tuning

```
public string Tuning { get; init; }
```

Property Value

string

Type

The name of the tuning's type

```
public string Type { get; init; }
```

Property Value

string

Class TuningFolderUpdateResult

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents what writing a folder update did

```
public sealed record TuningFolderUpdateResult : IEquatable<TuningFolderUpdateResult>
```

Inheritance

object ← TuningFolderUpdateResult

Implements

IEquatable<TuningFolderUpdateResult>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningFolderUpdateResult(IReadOnlyList<TuningFolderUpdateWrite>, string?)

Represents what writing a folder update did

```
public TuningFolderUpdateResult(IReadOnlyList<TuningFolderUpdateWrite> Packages,
string? ReportPath)
```

Parameters

`Packages` IReadOnlyList<TuningFolderUpdateWrite>

What was done for each package that had changes to make

`ReportPath` string

Where the report was written, if it was

Properties

Packages

What was done for each package that had changes to make

```
public IReadOnlyList<TuningFolderUpdateWrite> Packages { get; init; }
```

Property Value

IReadOnlyList <TuningFolderUpdateWrite>

ReportPath

Where the report was written, if it was

```
public string? ReportPath { get; init; }
```

Property Value

string

Class TuningFolderUpdateWrite

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents what writing a folder update did for one of its packages

```
public sealed record TuningFolderUpdateWrite : IEquatable<TuningFolderUpdateWrite>
```

Inheritance

object ← TuningFolderUpdateWrite

Implements

IEquatable<TuningFolderUpdateWrite>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningFolderUpdateWrite(string, string?, IReadOnlyList<TuningPackageUpdateChange>, string?)

Represents what writing a folder update did for one of its packages

```
public TuningFolderUpdateWrite(string RelativePath, string? WrittenPath,
IReadOnlyList<TuningPackageUpdateChange> Changes, string? Skipped)
```

Parameters

`RelativePath` string

The path of the package within the folder

`WrittenPath` string

Where its updated copy was written, if one was

`Changes` IReadOnlyList<TuningPackageUpdateChange>

What was changed

`Skipped` string

Why no copy was written, if one was planned but not written

Properties

Changes

What was changed

```
public IReadOnlyList<TuningPackageUpdateChange> Changes { get; init; }
```

Property Value

IReadOnlyList<TuningPackageUpdateChange>

RelativePath

The path of the package within the folder

```
public string RelativePath { get; init; }
```

Property Value

string

Skipped

Why no copy was written, if one was planned but not written

```
public string? Skipped { get; init; }
```

Property Value

string

WrittenPath

Where its updated copy was written, if one was

```
public string? WrittenPath { get; init; }
```

Property Value

string

Class TuningMerge

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Merges a mod's changes to a tuning onto the game's new version of it: a three-way merge of the version the mod was made from (the base), the mod's version, and the game's new version

```
public static class TuningMerge
```

Inheritance

object ← TuningMerge

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

Elements are matched by name (the element of a variant by the variant's option), and list items are aligned, so that a changed item is not mistaken for one removed and another added: items are the same item when their XML is the same, or, for items with members, when at least half of their values are.

Where only one side changed an element, the merge takes that side's version. Where both changed it differently, or one removed an item the other changed, that is a conflict, resolved as the caller says (by default, the game's version). Items both sides added at the same place are all kept, the game's first, and an item both added is kept once (where the mod put it, if they put it in different places). Attributes merge the same way, so a change the mod made to the root element's attributes is kept.

Methods

Merge(string, string, string, Func<TuningMergeConflict, TuningMergeChoice>?)

Merges the changes the mod made in `modText` to `baseText` onto `gameText`, resolving each conflict as `resolve` says (by default, the game's version)

```
public static TuningMergeResult Merge(string baseText, string modText, string gameText,
```

```
  Func<TuningMergeConflict, TuningMergeChoice>? resolve = null)
```

Parameters

`baseText` string

The version of the tuning the mod was made from

`modText` string

The mod's version of the tuning

`gameText` string

The game's new version of the tuning

`resolve` Func<TuningMergeConflict, TuningMergeChoice>

Chooses a side for each conflict, or null to take the game's version every time

Returns

TuningMergeResult

Enum TuningMergeChoice

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies which side of a merge conflict a merge takes

```
public enum TuningMergeChoice
```

Fields

`Game = 0`

The merge takes the game's new version

`Mod = 1`

The merge takes the mod's version

Class TuningMergeConflict

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a place where a merge could not combine the mod's change with the game's

```
public sealed record TuningMergeConflict : IEquatable<TuningMergeConflict>
```

Inheritance

object ← TuningMergeConflict

Implements

IEquatable<TuningMergeConflict>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningMergeConflict(int, TuningMergeConflictKind, string, string?, string?, string?)

Represents a place where a merge could not combine the mod's change with the game's

```
public TuningMergeConflict(int Index, TuningMergeConflictKind Kind, string Path, string?
Base, string? Mod, string? Game)
```

Parameters

`Index` int

The position of the conflict among the merge's conflicts, which does not depend on how conflicts are resolved

`Kind` TuningMergeConflictKind

Why the changes could not be combined

`Path` string

The path of the element in the merged result, such as `loot_list[2].stat`, or of an attribute, such as `@c`

`Base` string

The element in the version the mod was made from (the XML of the element on one line, or an attribute's value), if it was there

`Mod` string

The element in the mod, if it is there

`Game` string

The element in the game's new version, if it is there

Properties

Base

The element in the version the mod was made from (the XML of the element on one line, or an attribute's value), if it was there

```
public string? Base { get; init; }
```

Property Value

string

Game

The element in the game's new version, if it is there

```
public string? Game { get; init; }
```

Property Value

string

Index

The position of the conflict among the merge's conflicts, which does not depend on how conflicts are resolved

```
public int Index { get; init; }
```

Property Value

int

Kind

Why the changes could not be combined

```
public TuningMergeConflictKind Kind { get; init; }
```

Property Value

TuningMergeConflictKind

Mod

The element in the mod, if it is there

```
public string? Mod { get; init; }
```

Property Value

string

Path

The path of the element in the merged result, such as `loot_list[2].stat`, or of an attribute, such as `@c`

```
public string Path { get; init; }
```

Property Value

string

Enum TuningMergeConflictKind

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies why a merge could not combine the mod's change with the game's

```
public enum TuningMergeConflictKind
```

Fields

`BothChanged = 0`

The mod and the game both changed the element, differently

`GameRemovedModChanged = 2`

The game removed the element, and the mod changed it

`ModRemovedGameChanged = 1`

The mod removed the element, and the game changed it

`TooLarge = 3`

The list has too many items for its items to be aligned

Class TuningMergeResult

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents the result of a merge of a mod's changes onto the game's new version of a tuning

```
public sealed record TuningMergeResult : IEquatable<TuningMergeResult>
```

Inheritance

object ← TuningMergeResult

Implements

IEquatable<TuningMergeResult>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningMergeResult(string, IReadOnlyList<TuningMergeConflict>)

Represents the result of a merge of a mod's changes onto the game's new version of a tuning

```
public TuningMergeResult(string Text, IReadOnlyList<TuningMergeConflict> Conflicts)
```

Parameters

`Text` string

The merged tuning (canonical text, see TuningXml), with each conflict resolved as the merge was told to resolve it

`Conflicts` IReadOnlyList<TuningMergeConflict>

The places where the mod's changes could not be combined with the game's

Properties

Conflicts

The places where the mod's changes could not be combined with the game's

```
public IReadOnlyList<TuningMergeConflict> Conflicts { get; init; }
```

Property Value

IReadOnlyList <TuningMergeConflict>

Text

The merged tuning (canonical text, see TuningXml), with each conflict resolved as the merge was told to resolve it

```
public string Text { get; init; }
```

Property Value

string

Class TuningNode

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents an element of a tuning document, as understood through its tuning description

```
public sealed class TuningNode
```

Inheritance

object ← TuningNode

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Properties

Description

Gets the tunable description of the element, if one was found

```
public TunableDescription? Description { get; }
```

Property Value

TunableDescription

ElementName

Gets the name of the element (such as `T`, `L`, `U`, `V` or `E`)

```
public string ElementName { get; }
```

Property Value

string

Enum

Gets the enumeration of the value, if ValueKind is Enum and it was found

```
public TuningDescriptionClass? Enum { get; }
```

Property Value

TuningDescriptionClass

EnumValue

Gets the numeric value of the enumeration member, if it was found

```
public long? EnumValue { get; }
```

Property Value

long?

Line

Gets the 1-based line on which the element starts in the canonical text

```
public int Line { get; }
```

Property Value

int

Members

Gets the tunables the tuning descriptions declare for the element, in the order they declare them, if it is a tuple; otherwise, an empty list

```
public IReadOnlyList<TunableDescription> Members { get; }
```

Property Value

IReadOnlyList<TunableDescription>

Option

Gets the option the element chooses (its `t` attribute), if it is a variant

```
public string? Option { get; }
```

Property Value

string

Path

Gets the path of the element from the root, such as `loot_list[2].stat`

```
public string Path { get; }
```

Property Value

string

ReferenceInstance

Gets the instance the value refers to, if ValueKind is Reference and it is numeric

```
public ulong? ReferenceInstance { get; }
```

Property Value

ulong?

ReferenceName

Gets the name the value refers to, if ValueKind is Reference and it is not numeric

```
public string? ReferenceName { get; }
```

Property Value

string

ReferenceType

Gets the resource type the value refers to, if ValueKind is Reference

```
public ResourceType? ReferenceType { get; }
```

Property Value

ResourceType?

ReferenceTypeName

Gets the name of the type the value refers to (as the tuning description names it, such as `buff`), if ValueKind is Reference

```
public string? ReferenceTypeName { get; }
```

Property Value

string

ResourceKey

Gets the resource key of the value, if ValueKind is ResourceKey

```
public ResourceKey? ResourceKey { get; }
```

Property Value

ResourceKey?

StringKey

Gets the key of the localized string, if ValueKind is LocalizedString

```
public uint? StringKey { get; }
```

Property Value

uint?

Value

Gets the text value of the element, if it has one

```
public string? Value { get; }
```

Property Value

string

ValueEndColumn

Gets the 1-based column at which the element's value ends (exclusive), or 0 if the value is not on Line

```
public int ValueEndColumn { get; }
```

Property Value

int

ValueKind

Gets what the value of the element means

```
public TuningValueKind ValueKind { get; }
```

Property Value

TuningValueKind

ValueStartColumn

Gets the 1-based column at which the element's value starts, or 0 if the value is not on Line

```
public int ValueStartColumn { get; }
```

Property Value

int

Class TuningPackageAddedSimData

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents the SimData of tuning a package adds (rather than overrides), checked against SimData generated from the tuning with the tuning descriptions of the game version the package is being updated for

```
public sealed class TuningPackageAddedSimData
```

Inheritance

object ← TuningPackageAddedSimData

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The tuning descriptions say which columns the game version expects, so SimData made for an older version can be found missing columns without the game having SimData for the tuning.

Properties

ExtraColumns

Gets the columns, as `Schema.column`, the SimData has that the tuning descriptions don't give

```
public IReadOnlyList<string> ExtraColumns { get; }
```

Property Value

IReadOnlyList<string>

HasScriptBeside

Gets whether the package has a script mod beside it, in the same folder, which could add tunables to the game that no tuning description has

```
public bool HasScriptBeside { get; }
```

Property Value

bool

Key

Gets the key of the SimData

```
public ResourceKey Key { get; }
```

Property Value

ResourceKey

LeftOutTunables

Gets the tunables, with values, that the tuning leaves out (so the tuning descriptions' defaults apply) but the SimData has other values for, which adding to the tuning makes the SimData generated from it have

```
public IReadOnlyList<string> LeftOutTunables { get; }
```

Property Value

IReadOnlyList<string>

Message

Gets what is wrong with the SimData, for the creator

```
public string Message { get; }
```

Property Value

string

MismatchedHashes

Gets the schemas whose hashes don't match their columns

```
public IReadOnlyList<string> MismatchedHashes { get; }
```

Property Value

IReadOnlyList<string>

MissingColumns

Gets the columns, as `Schema.column`, the tuning descriptions give that the SimData doesn't have

```
public IReadOnlyList<string> MissingColumns { get; }
```

Property Value

IReadOnlyList<string>

MistypedColumns

Gets the columns, as `Schema.column`, whose types aren't what the tuning descriptions give

```
public IReadOnlyList<string> MistypedColumns { get; }
```

Property Value

IReadOnlyList<string>

Name

Gets the name of the tuning

```
public string? Name { get; }
```

Property Value

string

Problems

Gets what is wrong with the SimData

```
public TuningPackageAddedSimDataProblems Problems { get; }
```

Property Value

TuningPackageAddedSimDataProblems

TuningKey

Gets the key of the tuning

```
public ResourceKey TuningKey { get; }
```

Property Value

ResourceKey

Methods

Plan(TuningPackageAddedSimDataFixes)

Gets what the specified `fixes` do to the SimData

```
public TuningPackageAddedSimDataPlan Plan(TuningPackageAddedSimDataFixes fixes)
```

Parameters

`fixes` TuningPackageAddedSimDataFixes

Returns

TuningPackageAddedSimDataPlan

Enum TuningPackageAddedSimDataFixes

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies which fixes writing an updated package makes to the SimData of tuning it adds, and to that tuning

```
[Flags]
public enum TuningPackageAddedSimDataFixes
```

Fields

`AddMissingColumns = 1`

The columns the tuning descriptions give but the SimData doesn't have are added, with values generated from the tuning, keeping every value the SimData has

`All = AddMissingColumns | CorrectColumnTypes | RemoveExtraColumns | CorrectSchemaHashes | MatchTuning | AmendTuning`

All of the fixes

`AmendTuning = 32`

Tunables the tuning leaves out, but the SimData has other values for than their defaults, are added to the tuning with those values, when SimData generated from the amended tuning has them and nothing else changes

`CorrectColumnTypes = 2`

Columns whose types aren't what the tuning descriptions give get those types, with values generated from the tuning

`CorrectSchemaHashes = 8`

Schema hashes that don't match their columns are recomputed

`MatchTuning = 16`

The SimData is replaced with SimData generated from the tuning, when every column of it can be generated reliably

`None = 0`

The SimData is left as it is

`RemoveExtraColumns = 4`

Columns the tuning descriptions don't give are removed, unless the package has a script mod beside it, which could add those tunables to the game

Remarks

A value is only ever taken from SimData generated from the mod's tuning when the generator reproduces that column in every SimData of the game's that has it. The SimData is changed only when, after the fixes, it has every column the tuning descriptions give, with their types.

Class TuningPackageAddedSimDataPlan

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents what a set of fixes does to the SimData of tuning a package adds, and to the tuning

```
public sealed record TuningPackageAddedSimDataPlan :
IEquatable<TuningPackageAddedSimDataPlan>
```

Inheritance

object ← TuningPackageAddedSimDataPlan

Implements

IEquatable<TuningPackageAddedSimDataPlan>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningPackageAddedSimDataPlan(TuningPackageAddedSimDataProblems, TuningPackageAddedSimDataProblems, string)

Represents what a set of fixes does to the SimData of tuning a package adds, and to the tuning

```
public TuningPackageAddedSimDataPlan(TuningPackageAddedSimDataProblems Fixed,
TuningPackageAddedSimDataProblems Remaining, string Message)
```

Parameters

`Fixed` TuningPackageAddedSimDataProblems

The problems the fixes correct

`Remaining` TuningPackageAddedSimDataProblems

The problems left after the fixes

`Message` string

What the fixes do, or why they do nothing, for the creator

Properties

ChangesSimData

Gets whether the fixes change the SimData

```
public bool ChangesSimData { get; }
```

Property Value

bool

ChangesTuning

Gets whether the fixes change the tuning

```
public bool ChangesTuning { get; }
```

Property Value

bool

Fixed

The problems the fixes correct

```
public TuningPackageAddedSimDataProblems Fixed { get; init; }
```

Property Value

TuningPackageAddedSimDataProblems

Message

What the fixes do, or why they do nothing, for the creator

```
public string Message { get; init; }
```

Property Value

string

Remaining

The problems left after the fixes

```
public TuningPackageAddedSimDataProblems Remaining { get; init; }
```

Property Value

TuningPackageAddedSimDataProblems

Enum TuningPackageAddedSimDataProblems

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies what is wrong with the SimData of tuning a package adds, by the tuning descriptions of the game version it is being updated for

```
[Flags]
public enum TuningPackageAddedSimDataProblems
```

Fields

`ExtraColumns = 4`

It has columns the tuning descriptions don't give

`MismatchedHashes = 8`

Some of its schema hashes don't match their columns

`MissingColumns = 1`

The tuning descriptions give the SimData columns it doesn't have

`MistypedColumns = 2`

Some of its columns have other types than the tuning descriptions give

`None = 0`

Nothing is wrong

`Unchecked = 32`

It couldn't be checked, because it couldn't be read or SimData couldn't be generated from its tuning

`Values = 16`

Some of its values aren't what its tuning makes

Class TuningPackageOverride

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents a mod's override of a game tuning, and what a game version means for it

```
public sealed class TuningPackageOverride
```

Inheritance

object ← TuningPackageOverride

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object)

Properties

Base

Gets the snapshot whose version of the tuning the mod was made from: the one the creator declared (see BaseIsDeclared), or else the one with the fewest differences from the override (see BaseIsAmbiguous), the newest with that version

```
public TuningArchiveSnapshot? Base { get; }
```

Property Value

TuningArchiveSnapshot

BaseDisagrees

Gets whether the base is the version the creator said the package was last updated for, but another version of the tuning is closer to the override

```
public bool BaseDisagrees { get; }
```

Property Value

bool

BaseDistance

Gets the number of differences between BaseText and ModText

```
public int BaseDistance { get; }
```

Property Value

int

BaseIsAmbiguous

Gets whether a version of the tuning with different content is as close to the override as the base (the base is the oldest such version, so that a change the game also made shows as a conflict)

```
public bool BaseIsAmbiguous { get; }
```

Property Value

bool

BaseIsDeclared

Gets whether the base is the version the creator said the package was last updated for, rather than inferred

```
public bool BaseIsDeclared { get; }
```

Property Value

bool

BaseIsUncertain

Gets whether the base may not be what the mod was made from: it was inferred and is ambiguous or may be older than the archive, or it was declared and another version is closer

```
public bool BaseIsUncertain { get; }
```

Property Value

bool

BaseMayBeOlder

Gets whether the mod may have been made from a version older than any in the archive: the archive's oldest snapshot already has the base's version of the tuning, and the override differs from it

```
public bool BaseMayBeOlder { get; }
```

Property Value

bool

Remarks

When it was, some of what look like the mod's changes are the game's changes since that older version, undone, and a merge carries them forward as if the mod had made them.

BaseText

Gets the base's version of the tuning (canonical text, see TuningXml)

```
public string? BaseText { get; }
```

Property Value

string

GameEntry

Gets the tuning in the game's new version

```
public TuningArchiveEntry? GameEntry { get; }
```

Property Value

TuningArchiveEntry

GameText

Gets the game's new version of the tuning (canonical text)

```
public string? GameText { get; }
```

Property Value

string

Key

Gets the key of the override in the package

```
public ResourceKey Key { get; }
```

Property Value

ResourceKey

Merge

Gets the merge of the mod's changes onto the game's new version, with every conflict resolved as the game's version, if the game changed the tuning since the base

```
public TuningMergeResult? Merge { get; }
```

Property Value

TuningMergeResult

Message

Gets what the outcome means, for the creator

```
public string Message { get; }
```

Property Value

string

ModText

Gets the override (canonical text), if it could be read

```
public string? ModText { get; }
```

Property Value

string

Name

Gets the name of the tuning

```
public string? Name { get; }
```

Property Value

string

Outcome

Gets what the game's new version means for the override

```
public TuningPackageOverrideOutcome Outcome { get; }
```

Property Value

TuningPackageOverrideOutcome

SimData

Gets whether the mod's SimData for the override is what the tuning in GetRecommendedTuning() needs, if SimData was checked

```
public TuningPackageSimDataState SimData { get; }
```

Property Value

TuningPackageSimDataState

SimDataMessage

Gets what SimData means, for the creator

```
public string? SimDataMessage { get; }
```

Property Value

string

Methods

Remerge(Func<TuningMergeConflict, TuningMergeChoice>)

Merges the mod's changes onto the game's new version again, resolving each conflict as `resolve` says

```
public TuningMergeResult Remerge(Func<TuningMergeConflict, TuningMergeChoice> resolve)
```

Parameters

`resolve` Func<TuningMergeConflict, TuningMergeChoice>

Returns

TuningMergeResult

Exceptions

InvalidOperationException

The game did not change the tuning since the base, so there is nothing to merge

ToString()

Returns a string that represents the current object.

```
public override string ToString()
```

Returns

string

A string that represents the current object.

Enum TuningPackageOverrideOutcome

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies what a game version means for a mod's override of a tuning

```
public enum TuningPackageOverrideOutcome
```

Fields

`Conflicted = 2`

The game changed the tuning, and some of the mod's changes conflict with the game's

`MatchesGame = 3`

The override is the same as the game's new version, so it does nothing

`MatchesOlderGame = 4`

The override is the same as an older version of the game's, so all it does is undo the game's changes since then

`Merged = 1`

The game changed the tuning, and the mod's changes merge onto the game's new version without conflicts

`RemovedFromGame = 5`

The game's new version no longer has the tuning

`Unreadable = 6`

The override could not be read as tuning

`UpToDate = 0`

The game has not changed the tuning since the version the mod was made from, so the override needs nothing

Enum TuningPackageSimDataChanges

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies which changes writing an updated package makes to its SimData

```
[Flags]
public enum TuningPackageSimDataChanges
```

Fields

`Add = 4`

Generated SimData is added for tuning whose changes reach SimData but has none (Missing), which makes those changes take effect in the game for the first time

`All = Replace | Remove | Add`

All of the changes

`None = 0`

The SimData is left as it is

`Remove = 2`

SimData the game's own covers is removed (Unneeded), as is the SimData of a removed override

`Replace = 1`

SimData that isn't what its tuning needs is replaced with generated SimData that is (Stale)

Enum TuningPackageSimDataState

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies whether a mod's SimData for an override is what the override's tuning needs in the game's new version

```
public enum TuningPackageSimDataState
```

Fields

`Current = 1`

The mod's SimData (or the game's, if the mod has none) is what the tuning needs

`Missing = 3`

The mod has no SimData, but the tuning's changes reach SimData, and SimData that has them was generated

`NotChecked = 0`

SimData was not checked

`Stale = 2`

The mod's SimData is not what the tuning needs, and SimData that is was generated

`Unneeded = 4`

The mod has SimData, but the tuning's changes don't reach SimData, so the game's SimData is what the tuning needs

`Unverifiable = 5`

The tuning's changes may reach SimData, but SimData that could be trusted could not be generated

Class TuningPackageUpdate

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Finds a mod package's overrides of game tuning, works out what a game version means for each, and writes an updated copy of the package

```
public sealed class TuningPackageUpdate
```

Inheritance

object ← TuningPackageUpdate

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

For each override, the base (the game version the mod was most likely made from) is the archive snapshot, up to the target, whose version of the tuning has the fewest differences from the override. When versions with different content are as close, the base is the oldest of them, because they tie when the mod changed what the game later changed, and the older base shows that as a conflict rather than taking the mod's version as up to date. If the game changed the tuning between the base and the target, the mod's changes are merged onto the target's version (see TuningMerge).

The package is never changed: WriteAsync(string, IReadOnlyDictionary<ResourceKey, string?>, TuningPackageSimDataChanges, TuningCommenter?, CancellationToken) writes a new one, copying every resource it is not told to change.

Properties

ChecksSimData

Gets whether the package's SimData was checked against what its overrides need, and can be made what they need when the updated package is written

```
public bool ChecksSimData { get; }
```

Property Value

bool

Overrides

Gets the package's overrides of game tuning

```
public IReadOnlyList<TuningPackageOverride> Overrides { get; }
```

Property Value

IReadOnlyList<TuningPackageOverride>

PackagePath

Gets the path of the package

```
public string PackagePath { get; }
```

Property Value

string

Target

Gets the snapshot of the game version the package is being updated for

```
public TuningArchiveSnapshot Target { get; }
```

Property Value

TuningArchiveSnapshot

Methods

AnalyzeAsync(TuningArchiveFile, TuningArchiveSnapshot, string, TuningSimDataDrafter?, TuningArchiveSnapshot?, IProgress<TuningArchiveProgress>?, CancellationToken)

Finds the overrides of game tuning in the package at the specified `packagePath` and works out what the `target` game version in the specified `archive` means for each (🏃)

```
public static Task<TuningPackageUpdate> AnalyzeAsync(TuningArchiveFile archive,
TuningArchiveSnapshot target, string packagePath, TuningSimDataDrafter? simData = null,
TuningArchiveSnapshot? declaredBase = null, IProgress<TuningArchiveProgress>? progress =
null, CancellationToken cancellationToken = default)
```

Parameters

`archive` TuningArchiveFile

The archive with the game versions

`target` TuningArchiveSnapshot

The game version to update the package for

`packagePath` string

The path of the package

`simData` TuningSimDataDrafter

A drafter for the `target` snapshot, to check the package's SimData against what its overrides need, or null not to

`declaredBase` TuningArchiveSnapshot

The game version the creator says the package was last updated for, to use as the base instead of inferring it (for each tuning that version has), or null to infer it

`progress` IProgress<TuningArchiveProgress>

Receives progress

`cancellationToken` CancellationToken

The token to monitor for cancellation requests

Returns

Task<TuningPackageUpdate>

GetRecommendedTuning()

Gets the tuning the update recommends writing: the merge, for each override whose changes merge onto the game's new version without conflicts

```
public IReadOnlyDictionary<ResourceKey, string?> GetRecommendedTuning()
```

Returns

IReadOnlyDictionary<ResourceKey, string>

Remarks

Nothing else is recommended, because only the creator knows whether an override that undoes the game's changes, or does nothing, is meant to.

WriteAsync(string, IReadOnlyDictionary<ResourceKey, string?>, TuningPackageSimDataChanges, TuningCommenter?, CancellationToken)

Writes a copy of the package to the specified `path` with the specified `tuning` in place of the overrides it has keys for (null to remove an override), and, if the update has a SimData drafter, the specified `simDataChanges` made to each override's SimData (🏃)

```
public Task<IReadOnlyList<TuningPackageUpdateChange>> WriteAsync(string path,
IReadOnlyDictionary<ResourceKey, string?> tuning, TuningPackageSimDataChanges simDataChanges
= TuningPackageSimDataChanges.All, TuningCommenter? commenter = null, CancellationToken
cancellationToken = default)
```

Parameters

`path` string

Where to write the updated package, which must not be the package's own path

`tuning` IReadOnlyDictionary<ResourceKey, string>

The tuning to write in place of overrides, by key, such as from GetRecommendedTuning()

`simDataChanges` TuningPackageSimDataChanges

Which changes to make to the SimData, to make it what each override's tuning needs

`commenter` TuningCommenter

Comments the tuning written in place of overrides, if specified

`cancellationToken` CancellationToken

The token to monitor for cancellation requests

Returns

Task<IReadOnlyList<TuningPackageUpdateChange>>

What was changed

Class TuningPackageUpdateChange

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents what writing an updated package did to a resource

```
public sealed record TuningPackageUpdateChange : IEquatable<TuningPackageUpdateChange>
```

Inheritance

object ← TuningPackageUpdateChange

Implements

IEquatable<TuningPackageUpdateChange>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningPackageUpdateChange(TuningPackageUpdateChangeKind, ResourceKey, string?, string)

Represents what writing an updated package did to a resource

```
public TuningPackageUpdateChange(TuningPackageUpdateChangeKind Kind, ResourceKey Key,
string? Name, string Message)
```

Parameters

`Kind` TuningPackageUpdateChangeKind

What was done

`Key` ResourceKey

The key of the resource

`Name` string

The name of the override's tuning

`Message` string

What was done, for the creator

Properties

Key

The key of the resource

```
public ResourceKey Key { get; init; }
```

Property Value

ResourceKey

Kind

What was done

```
public TuningPackageUpdateChangeKind Kind { get; init; }
```

Property Value

TuningPackageUpdateChangeKind

Message

What was done, for the creator

```
public string Message { get; init; }
```

Property Value

string

Name

The name of the override's tuning

```
public string? Name { get; init; }
```

Property Value

string

Enum TuningPackageUpdateChangeKind

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies what writing an updated package did to a resource

```
public enum TuningPackageUpdateChangeKind
```

Fields

`SimDataAdded = 3`

Generated SimData was added for the override

`SimDataRemoved = 4`

The override's SimData was removed, since the game's is what the tuning needs

`SimDataRepaired = 6`

The SimData of tuning the package adds was fixed

`SimDataReplaced = 2`

The override's SimData was replaced with generated SimData

`SimDataUnverifiable = 5`

The override's SimData was left as it was, since SimData that could be trusted could not be generated

`TuningAmended = 7`

Tuning the package adds was given values its SimData has that it left out

`TuningRemoved = 1`

The override was removed

`TuningReplaced = 0`

The override's tuning was replaced

Class TuningPythonDraft

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Drafts a Python script mod that makes changes to tuning as the game loads it, without overriding any tuning

```
public static class TuningPythonDraft
```

Inheritance

object ← TuningPythonDraft

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The script changes each tunable of instance tuning on the tuning's class after the game loads the tuning's data and before the game invokes any tuning callback, so callbacks and everything that reads tuning see the changes. The game loads module tuning when it imports the module, before script mods, so the script changes tunables of module tuning once the game has finished loading tuning (after it resolves the references module tuning defers), and leaves out tunables with a tuning callback, which has already run with the old value. It is defensive: each change is made only when the tunable still holds what it held in the archive (so a game update or another mod that changed it is never clobbered), list items are matched by their content rather than their position, a change that is already in effect is not made again, and each change fails on its own without affecting the rest. It writes what happened to each change to a log beside the mod. Changes to tunables the game also exports to SimData are left out, because the engine and the UI read SimData, which Python cannot change.

Methods

Generate(IReadOnlyList<TuningDraftTarget>, string, string, TuningCommenter?)

Drafts a script that makes the changes in the specified `targets`, for the specified `gameVersion`, logging to a file with the specified `logName` beside the mod, with the tuning in it commented by the specified `commenter`, if any

```
public static TuningDraftResult Generate(IReadOnlyList<TuningDraftTarget> targets, string
gameVersion, string logName, TuningCommenter? commenter = null)
```

Parameters

`targets` IReadOnlyList<TuningDraftTarget>

`gameVersion` string

`logName` string

`commenter` TuningCommenter

Returns

TuningDraftResult

Class TuningSemanticChange

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents an element that differs between two versions of a tuning document

```
public sealed record TuningSemanticChange : IEquatable<TuningSemanticChange>
```

Inheritance

object ← TuningSemanticChange

Implements

IEquatable<TuningSemanticChange>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningSemanticChange(TuningSemanticChangeKind, string, string?, string?, int, int)

Represents an element that differs between two versions of a tuning document

```
public TuningSemanticChange(TuningSemanticChangeKind Kind, string Path, string? OldValue,
string? NewValue, int OldLine, int NewLine)
```

Parameters

`Kind` TuningSemanticChangeKind

How the element differs

`Path` string

The path of the element from the root, such as `loot_list[2].stat` (list indices are those of the newer version, except for removed elements)

`OldValue` string

The element's value in the older version (the XML of the element, on one line, if it has children), if it was there

`NewValue` string

The element's value in the newer version (the XML of the element, on one line, if it has children), if it is there

`OldLine` int

The 1-based line of the element in the older version's canonical text, or 0 if it was not there

`NewLine` int

The 1-based line of the element in the newer version's canonical text, or 0 if it is not there

Properties

Kind

How the element differs

```
public TuningSemanticChangeKind Kind { get; init; }
```

Property Value

TuningSemanticChangeKind

NewLine

The 1-based line of the element in the newer version's canonical text, or 0 if it is not there

```
public int NewLine { get; init; }
```

Property Value

int

NewValue

The element's value in the newer version (the XML of the element, on one line, if it has children), if it is there

```
public string? NewValue { get; init; }
```

Property Value

string

OldLine

The 1-based line of the element in the older version's canonical text, or 0 if it was not there

```
public int OldLine { get; init; }
```

Property Value

int

OldValue

The element's value in the older version (the XML of the element, on one line, if it has children), if it was there

```
public string? OldValue { get; init; }
```

Property Value

string

Path

The path of the element from the root, such as `loot_list[2].stat` (list indices are those of the newer version, except for removed elements)

```
public string Path { get; init; }
```

Property Value

string

Enum TuningSemanticChangeKind

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies how an element differs between two versions of a tuning document

```
public enum TuningSemanticChangeKind
```

Fields

`Added = 0`

The element is only in the newer version

`Changed = 2`

The element's value (or, for a variant, its option) differs

`Removed = 1`

The element is only in the older version

Class TuningSemanticDiff

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Compares two versions of a tuning document element by element, matching elements by name and aligning list items, rather than line by line

```
public static class TuningSemanticDiff
```

Inheritance

object ← TuningSemanticDiff

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Compare(string, string)

Compares the `older` and `newer` versions of a tuning document (canonical text, see TuningXml)

```
public static IReadOnlyList<TuningSemanticChange> Compare(string older, string newer)
```

Parameters

`older` string

`newer` string

Returns

IReadOnlyList<TuningSemanticChange>

Enum TuningSimDataDraftKind

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies what an override of a tuning needs from its SimData

```
public enum TuningSimDataDraftKind
```

Fields

`Failed = 2`

The changes may reach the tuning's SimData, but SimData that could be trusted could not be generated

`Generated = 1`

The changes reach the tuning's SimData, and new SimData was generated

`Unaffected = 0`

The tuning has no SimData, or the changes do not reach it, so the game's SimData stays correct

Class TuningSimDataDraftResult

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Represents the SimData an override of a tuning needs

```
public sealed record TuningSimDataDraftResult : IEquatable<TuningSimDataDraftResult>
```

Inheritance

object ← TuningSimDataDraftResult

Implements

IEquatable<TuningSimDataDraftResult>

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

TuningSimDataDraftResult(TuningSimDataDraftKind, ResourceKey?, SimDataModel?, string)

Represents the SimData an override of a tuning needs

```
public TuningSimDataDraftResult(TuningSimDataDraftKind Kind, ResourceKey? Key, SimDataModel?
SimData, string Message)
```

Parameters

`Kind` TuningSimDataDraftKind

Whether the override needs new SimData, and whether it could be generated

`Key` ResourceKey?

The key of the tuning's SimData, if it has SimData

`SimData` SimDataModel

The generated SimData, if `Kind` is Generated

`Message` string

What happened, for the creator

Properties

Key

The key of the tuning's SimData, if it has SimData

```
public ResourceKey? Key { get; init; }
```

Property Value

ResourceKey?

Kind

Whether the override needs new SimData, and whether it could be generated

```
public TuningSimDataDraftKind Kind { get; init; }
```

Property Value

TuningSimDataDraftKind

Message

What happened, for the creator

```
public string Message { get; init; }
```

Property Value

string

SimData

The generated SimData, if `Kind` is Generated

```
public SimDataModel? SimData { get; init; }
```

Property Value

SimDataModel

Class TuningSimDataDrafter

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Generates the SimData that overrides of a snapshot's tuning need, trusting the result only when the generator reproduces the game's own SimData for the original tuning

```
public sealed class TuningSimDataDrafter
```

Inheritance

object ← TuningSimDataDrafter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The generator learns the entries of dynamic enumerations from the snapshot's module tuning, the names of its tuning, and, for each tuning, the column orders of its own SimData. Strings the game writes as null and the generator writes as empty count as the same, since the game and Sims 4 Studio read both as empty.

Methods

Create(TuningArchiveFile, TuningArchiveSnapshot, TuningDescriptionSet)

Creates a drafter for the tuning of the specified `snapshot` in the specified `archive`, which generates SimData with the specified `descriptions`

```
public static TuningSimDataDrafter Create(TuningArchiveFile archive, TuningArchiveSnapshot
snapshot, TuningDescriptionSet descriptions)
```

Parameters

`archive` TuningArchiveFile

`snapshot` TuningArchiveSnapshot

`descriptions` TuningDescriptionSet

Returns

TuningSimDataDrafter

Draft(TuningArchiveEntry, string)

Generates the SimData an override of the specified `tuning` with the specified `draftText` needs

```
public TuningSimDataDraftResult Draft(TuningArchiveEntry tuning, string draftText)
```

Parameters

`tuning` TuningArchiveEntry

`draftText` string

Returns

TuningSimDataDraftResult

Enum TuningValueKind

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Specifies what the value of an element of a tuning document means, according to its tuning description

```
public enum TuningValueKind
```

Fields

`Enum = 3`

The value is a member of an enumeration

`LocalizedString = 2`

The value is the key of a localized string

`None = 0`

The element has no value of its own, or its meaning is not known

`Reference = 1`

The value refers to another tuning or resource by instance (or, occasionally, by name)

`ResourceKey = 4`

The value is a resource key

`Scalar = 5`

The value is a number, Boolean or text

Class TuningXml

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Provides the canonical text form in which a tuning archive stores tuning and SimData documents

```
public static class TuningXml
```

Inheritance

object ← TuningXml

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

The canonical form is UTF-8 without a byte order mark, an XML declaration, two-space indentation and line feeds, with comments removed. The root element's attributes are in ordinal order (`c`, `i`, `m`, `n`, `s`), and every other element's are `n`, then `t`, then the rest in ordinal order. Documents with the same content therefore have the same canonical text however they were obtained (from the game's combined tuning or from files extracted by another tool).

Methods

FindElement(XElement, string)

Finds the element at the specified `path` (as Path spells it, such as `loot_list[2].stat`) below the specified `root`, or returns null if there is none

```
public static XElement? FindElement(XElement root, string path)
```

Parameters

`root` XElement

`path` string

Returns

XElement

GetCanonicalText(XDocument)

Gets the canonical text of the specified `document`

```
public static string GetCanonicalText(XDocument document)
```

Parameters

`document` XDocument

Returns

string

GetCanonicalText(XElement)

Gets the canonical text of a document whose root is the specified `root`

```
public static string GetCanonicalText(XElement root)
```

Parameters

`root` XElement

Returns

string

GetFragmentText(XElement)

Gets the canonical text of the specified `element` on one line, without an XML declaration

```
public static string GetFragmentText(XElement element)
```

Parameters

`element` XElement

Returns

string

GetResourceType(string)

Gets the type of the tuning resource for the specified tuning type name (the `i` attribute of an instance, or the name of a group in combined tuning), the way the game derives it

```
public static ResourceType GetResourceType(string typeName)
```

Parameters

`typeName` string

Returns

ResourceType

HaveSameFragmentText(XElement, XElement)

Gets whether the specified elements have the same canonical text on one line (what GetFragment Text(XElement) gets for each), without writing either

```
public static bool HaveSameFragmentText(XElement a, XElement b)
```

Parameters

`a` XElement

`b` XElement

Returns

bool

Parse(string)

Parses canonical text (or any tuning XML) into a document with line information

```
public static XDocument Parse(string text)
```

Parameters

`text` string

Returns

XDocument

Class TuningXmlInjectorDraft

Namespace: LlamaLogic.TuningArchive

Assembly: LlamaLogic.TuningArchive.dll

Drafts an XML Injector snippet (for Scumbumbo's XML Injector, version 4) that makes changes to tuning without overriding it

```
public static class TuningXmlInjectorDraft
```

Inheritance

object ← TuningXmlInjectorDraft

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Remarks

XML Injector can only add: interactions to objects, to the Sim object, to phones and to the relationship panel; interactions to affordance lists; loot actions to loot actions and random weighted loot; buffs to traits; states and state triggers to objects; name and object relationship components to objects that have none; and purchase list options and picker categories to purchase picker interactions. Any other change, and any removal, is reported as an error and left out.

Methods

Generate(IReadOnlyList<TuningDraftTarget>, string)

Drafts a snippet with the specified `name` that makes the changes in the specified `targets`

```
public static TuningDraftResult Generate(IReadOnlyList<TuningDraftTarget> targets,
string name)
```

Parameters

`targets` IReadOnlyList<TuningDraftTarget>

`name` string

Returns

TuningDraftResult